

# Intermediate Code Generation

## using Syntax-Directed Definitions (SDD)

## and Syntax-Directed Translation (SDT)

Lecture Notes based on *Compilers: Principles, Techniques, and Tools* (the “Dragon Book”),  
2nd Edition, Aho, Lam, Sethi, Ullman – Chapters 5 and 6

### Suggested Lecture Plan (3–4 lectures)

- Lecture 1** Role of intermediate code; forms of IR: syntax trees, DAGs, three-address code, quadruples, triples, indirect triples.
- Lecture 2** *Syntax-directed definitions and translation, from scratch*: why syntax alone is not enough, attributes, semantic rules, synthesized and inherited attributes, dependency graphs, S- and L-attributed definitions; then SDTs: actions inside productions, postfix translation, marker nonterminals.
- Lecture 3** Translation of expressions and assignments, array references, type checking and conversion.
- Lecture 4** Boolean expressions and short-circuit code, flow-of-control statements, back-patching, switch statements and procedure calls; revision and exercises.

## Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction: Why Intermediate Code?</b>                      | <b>4</b>  |
| 1.1      | Advantages of using an intermediate representation . . . . .     | 4         |
| <b>2</b> | <b>Variants of Syntax Trees and Intermediate Representations</b> | <b>4</b>  |
| 2.1      | Syntax trees (abstract syntax trees) . . . . .                   | 4         |
| 2.2      | Directed acyclic graphs (DAGs) for expressions . . . . .         | 4         |
| 2.2.1    | The value-number method for constructing DAGs . . . . .          | 5         |
| 2.3      | Three-address code (TAC) . . . . .                               | 8         |
| 2.3.1    | Generating three-address code from a DAG . . . . .               | 8         |
| 2.3.2    | Addresses and instructions . . . . .                             | 9         |
| 2.3.3    | Representations of three-address code . . . . .                  | 9         |
| 2.3.4    | Static single-assignment (SSA) form – a brief note . . . . .     | 10        |
| <b>3</b> | <b>Syntax-Directed Definitions (SDD)</b>                         | <b>11</b> |

|          |                                                                                 |           |
|----------|---------------------------------------------------------------------------------|-----------|
| 3.1      | Why do we need them? Syntax is not enough . . . . .                             | 11        |
| 3.2      | Attributes . . . . .                                                            | 11        |
| 3.3      | Semantic rules: a first tiny example . . . . .                                  | 12        |
| 3.4      | Formal definition . . . . .                                                     | 12        |
| 3.5      | Synthesized and inherited attributes . . . . .                                  | 12        |
| 3.6      | Annotated parse trees and dependency graphs . . . . .                           | 14        |
| 3.7      | Classes of SDDs that are easy to implement . . . . .                            | 15        |
| 3.8      | SDD for constructing syntax trees . . . . .                                     | 15        |
| <b>4</b> | <b>Syntax-Directed Translation Schemes (SDT)</b>                                | <b>16</b> |
| 4.1      | From specification to implementation . . . . .                                  | 16        |
| 4.2      | Definition and the meaning of an action's position . . . . .                    | 16        |
| 4.3      | Postfix translation schemes . . . . .                                           | 17        |
| 4.4      | SDTs with actions inside productions . . . . .                                  | 18        |
| 4.5      | Eliminating left recursion from SDTs and marker nonterminals . . . . .          | 19        |
| 4.6      | Converting an L-attributed SDD into an SDT . . . . .                            | 19        |
| <b>5</b> | <b>Translation of Expressions and Assignments</b>                               | <b>19</b> |
| 5.1      | SDD for three-address code . . . . .                                            | 19        |
| 5.2      | Incremental translation (SDT with <code>gen</code> emitting directly) . . . . . | 20        |
| 5.3      | Addressing array elements . . . . .                                             | 21        |
| 5.4      | Translation of array references . . . . .                                       | 21        |
| <b>6</b> | <b>Type Checking and Type Conversion</b>                                        | <b>22</b> |
| 6.1      | Type conversion (coercion) . . . . .                                            | 22        |
| <b>7</b> | <b>Control Flow: Boolean Expressions and Statements</b>                         | <b>23</b> |
| 7.1      | Short-circuit (jumping) code . . . . .                                          | 23        |
| 7.2      | Flow-of-control statements: SDD with inherited labels . . . . .                 | 24        |
| 7.3      | SDD for boolean expressions (jumping code) . . . . .                            | 24        |
| 7.4      | Avoiding redundant gotos (fall-through) . . . . .                               | 26        |
| <b>8</b> | <b>Backpatching</b>                                                             | <b>27</b> |
| 8.1      | The problem and the idea . . . . .                                              | 27        |
| 8.2      | Boolean expressions . . . . .                                                   | 27        |
| 8.3      | Flow-of-control statements . . . . .                                            | 28        |
| 8.4      | Comparison: inherited labels vs. backpatching . . . . .                         | 30        |
| <b>9</b> | <b>Switch Statements and Procedure Calls (Brief)</b>                            | <b>30</b> |

|                                     |           |
|-------------------------------------|-----------|
| 9.1 Switch statement . . . . .      | 30        |
| 9.2 Procedure calls . . . . .       | 30        |
| <b>10 A Complete Worked Example</b> | <b>31</b> |
| <b>11 Summary and Key Takeaways</b> | <b>32</b> |
| <b>12 Practice Problems</b>         | <b>32</b> |

# 1 Introduction: Why Intermediate Code?

In the *analysis-synthesis* model of a compiler, the **front end** analyses the source program and produces an **intermediate representation (IR)**; the **back end** synthesises the target program from that IR.

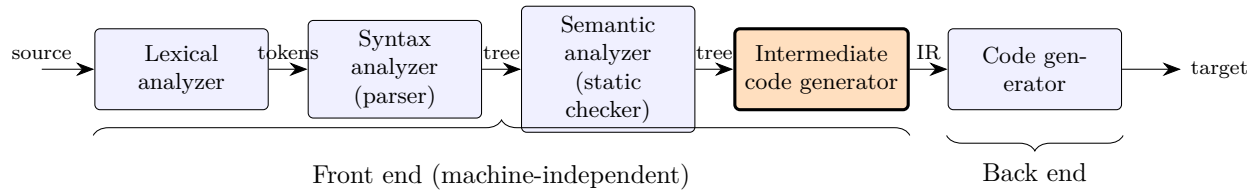


Figure 1: Position of the intermediate code generator in a compiler. The shaded (thick) box is the topic of these notes. In practice, static checking and intermediate code generation are often done together in one pass.

## 1.1 Advantages of using an intermediate representation

- **Retargeting.** A compiler for  $m$  languages and  $n$  machines needs only  $m$  front ends plus  $n$  back ends ( $m+n$  pieces instead of  $m \times n$  separate compilers) if they share one IR. For example, 3 languages and 4 machines need  $3 + 4 = 7$  pieces instead of 12 compilers.
- **Machine-independent optimisation.** Optimisations (common sub-expression elimination, constant folding, loop-invariant code motion, ...) are applied once on the IR.
- **Simplicity.** Complex source constructs are lowered to a few simple operations, making code generation easier.

### Static checking vs. intermediate code

Static checking includes type checking and syntactic checks that the grammar cannot enforce (e.g., **break** must be inside a loop). Dragon Book Chapter 6 treats type checking and intermediate-code generation together, because both are done by attaching attributes and actions to the grammar.

# 2 Variants of Syntax Trees and Intermediate Representations

## 2.1 Syntax trees (abstract syntax trees)

A **syntax tree** is a condensed form of the parse tree: operators and keywords appear as *interior nodes*, and operands as children. Chains of single productions (such as  $E \rightarrow T \rightarrow F \rightarrow \text{id}$ ) disappear, and so do punctuation tokens such as parentheses. The tree keeps only what later phases need: *which operation is applied to which operands*.

### Example: parse tree vs. syntax tree for $a + b * c$

Take the usual grammar  $E \rightarrow E + T \mid T$ ,  $T \rightarrow T * F \mid F$ ,  $F \rightarrow \text{id}$ .

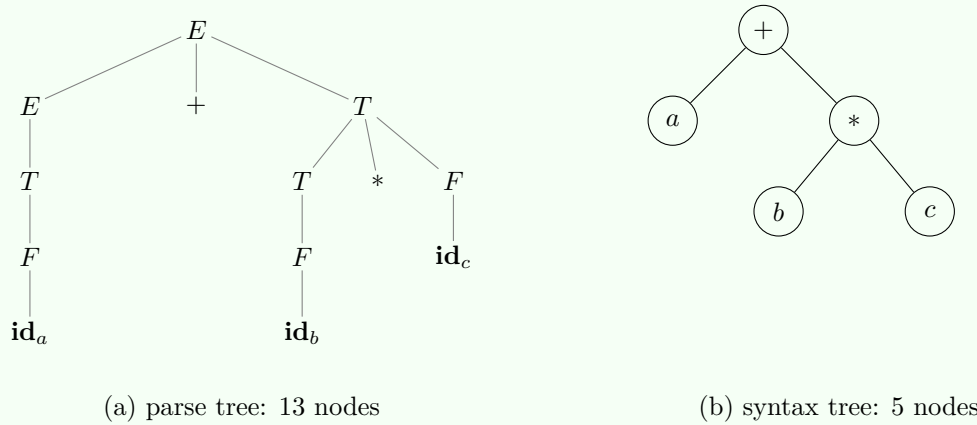


Figure 2: Parse tree and syntax tree of  $a + b * c$ . Precedence survives in the shape of the syntax tree:  $*$  is below  $+$ , so it is evaluated first.

Most parse-tree nodes only record *how the grammar was written* (the chains  $E \rightarrow T \rightarrow F$ ); the syntax tree has 5 nodes. Parentheses never appear in a syntax tree:  $(a + b) * c$  simply has the  $+$  node below the  $*$  node.

## 2.2 Directed acyclic graphs (DAGs) for expressions

A **DAG** identifies the *common sub-expressions*: a node is shared if the same sub-expression appears more than once. It shows the same information as a syntax tree but is more compact, and it directly suggests how to generate efficient code that computes a common sub-expression only once.

### Example: tree vs. DAG for $a + a * (b - c) + (b - c) * d$

Figure 2 shows the syntax tree (left) and the DAG (right). In the DAG, the leaf  $a$  and the sub-expression  $b - c$  are each shared by two parents.

**Why arrows?** A DAG is a *directed* acyclic graph. Each edge points from an operator node to one of its operands (“this operation uses that value”), so a node with two incoming arrows is a value that is used twice. There is no cycle because an expression can never be an operand of itself. In a tree the direction is the same but is left implicit, since every node has exactly one parent.

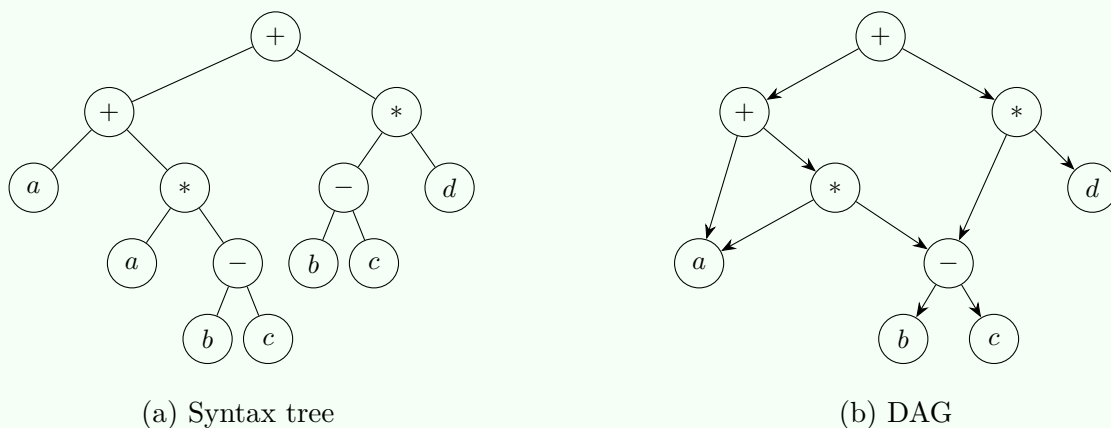


Figure 3: Syntax tree and DAG for  $a + a * (b - c) + (b - c) * d$ . In the DAG every edge is an arrow from an operator to one of its operands.

### 2.2.1 The value-number method for constructing DAGs

**The problem.** While building the tree for an expression, we want to *reuse* a node if the same sub-expression has already been built. To do this we need a fast way to answer: “Have I already created a node for  $b - c$ ?” The **value-number method** gives a simple answer. Here we assume the *syntax tree* of the expression has already been built (Figure 2(a)). We convert it into a DAG by visiting the tree nodes in **post-order** (left subtree, right subtree, then the node itself), so that the children of a node are always handled before the node.

**The idea.** Keep all nodes in an **array of records**. Each record describes one node of the DAG, and the *position (index) of the record in the array is the node’s value number*. The value number is therefore just the node’s *name*: “node 4” means “the record at position 4”.

#### What the numbers mean

- **Value number** = the index of a record in the array = the identity of one DAG node. It is *not* the numeric value of the expression (the number 4 does not mean the expression evaluates to 4).
- A **leaf record** stores the kind of leaf and what it is: (**id**, pointer to symbol-table entry) for a variable, or (**num**, value) for a constant.
- An **interior record** has the form (*op*, *left*, *right*). Here *op* is the operator, and **left and right are the value numbers of the two child nodes**, i.e., they are *pointers* to other records (the operands).

**Example.** The record  $(*, 1, 4)$  means: “a node for the operator  $*$  whose left child is node 1 and whose right child is node 4”. Looking up the array, node 1 is the leaf  $a$  and node 4 is  $b - c$ ; so this record stands for  $a * (b - c)$ .

#### Algorithm: finding or creating a node

To create a node with signature  $\langle op, l, r \rangle$  (where  $l$  and  $r$  are value numbers of the children that were already created):

1. **Search** the array for a record with exactly the same *op*, *left* =  $l$ , and *right* =  $r$ .
2. If such a record **exists**, do not create anything; **return its value number**. (This is the common sub-expression being shared.)
3. Otherwise **append** a new record  $(op, l, r)$  at the end of the array and **return its index** as the new value number.

(For speed, a hash table keyed on  $(op, l, r)$  replaces the linear search in step 1.)

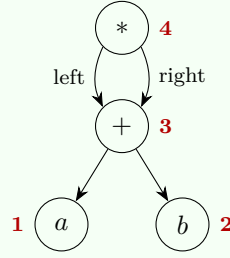
**Why does comparing only  $(op, l, r)$  work?** Two nodes represent the same expression exactly when they have the same operator and the same children. Children are the same exactly when they have the same value numbers (because every distinct sub-expression gets exactly one number). So, by building from the leaves upward, equal sub-expressions always get equal numbers.

#### A second example: $(a + b) * (a + b)$ and the commutativity trap

Post-order requests and the records they create:

1. leaf  $a \Rightarrow$  new record 1; leaf  $b \Rightarrow$  new record 2;  $\langle +, 1, 2 \rangle \Rightarrow$  new record 3.
2. Second copy: leaf  $a \Rightarrow$  found (1); leaf  $b \Rightarrow$  found (2);  $\langle +, 1, 2 \rangle \Rightarrow$  **found (3)**, shared.
3.  $\langle *, 3, 3 \rangle \Rightarrow$  new record 4 (both children are the same node).

The DAG has 4 nodes (the tree has 7) and gives  $t1 = a + b$ ,  $t2 = t1 * t1$ . Note that the  $*$  node has *two* arrows into the same node: its left and its right operand are one and the same value.



Now try  $(a + b) * (b + a)$ : the second sum is  $\langle +, 2, 1 \rangle$ , which is *not* equal to  $\langle +, 1, 2 \rangle$ , so a new node is created and  $a + b$  is computed twice. To avoid this, a compiler stores the children of a commutative operator in a canonical order (say, smaller value number first); then  $\langle +, 2, 1 \rangle$  becomes  $\langle +, 1, 2 \rangle$  and is shared.

### Limits of the method

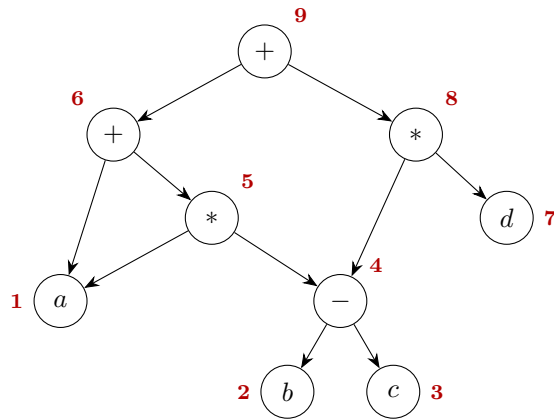
The test “same *op* and same children” finds only *syntactically identical* sub-expressions (up to the operand-order trick above). It does not know that  $x * 2$  and  $x + x$  are equal; that needs algebraic simplification, a separate optimisation.

**Step-by-step example:**  $a + a * (b - c) + (b - c) * d$ . We walk the syntax tree of Figure 2(a) in post-order:  $a$ ,  $a$ ,  $b$ ,  $c$ , then  $-$ ,  $*$ ,  $+$  of the left part, then  $b$ ,  $c$ ,  $-$ ,  $d$ ,  $*$ , and finally the root  $+$ . At every tree node we ask the array for the matching DAG node. Table 1 shows each request and what the algorithm does.

| Step | Request                                         | Found in array?      | Action                            | Returns |
|------|-------------------------------------------------|----------------------|-----------------------------------|---------|
| 1    | leaf $a$                                        | no                   | create record 1: $(\text{id}, a)$ | 1       |
| 2    | leaf $a$ (2nd time)                             | <b>yes, record 1</b> | nothing created                   | 1       |
| 3    | leaf $b$                                        | no                   | create record 2: $(\text{id}, b)$ | 2       |
| 4    | leaf $c$                                        | no                   | create record 3: $(\text{id}, c)$ | 3       |
| 5    | $\langle -, 2, 3 \rangle$ ( $b - c$ )           | no                   | create record 4: $(-, 2, 3)$      | 4       |
| 6    | $\langle *, 1, 4 \rangle$ ( $a * (b - c)$ )     | no                   | create record 5: $(*, 1, 4)$      | 5       |
| 7    | $\langle +, 1, 5 \rangle$ ( $a + a * (b - c)$ ) | no                   | create record 6: $(+, 1, 5)$      | 6       |
| 8    | leaf $b$ (2nd time)                             | <b>yes, record 2</b> | nothing created                   | 2       |
| 9    | leaf $c$ (2nd time)                             | <b>yes, record 3</b> | nothing created                   | 3       |
| 10   | $\langle -, 2, 3 \rangle$ ( $b - c$ again)      | <b>yes, record 4</b> | nothing created – <i>shared!</i>  | 4       |
| 11   | leaf $d$                                        | no                   | create record 7: $(\text{id}, d)$ | 7       |
| 12   | $\langle *, 4, 7 \rangle$ ( $((b - c) * d)$ )   | no                   | create record 8: $(*, 4, 7)$      | 8       |
| 13   | $\langle +, 6, 8 \rangle$ (whole expression)    | no                   | create record 9: $(+, 6, 8)$      | 9       |

Table 1: Trace of the value-number method. Steps 2, 8, 9, 10 reuse existing nodes; step 10 is where the common sub-expression  $b - c$  is detected.

The result has only 9 nodes (a plain syntax tree would have 13: it repeats  $a$ ,  $b$ ,  $c$  and the whole  $b - c$  sub-tree). The final value number 9 identifies the root of the DAG.



**red number** = value number  
(index of the node's record  
in the array)

Arrows point from an  
operator to its operands.

Nodes 1 and 4 each have  
**two parents** (two  
incoming arrows): they  
are shared.

Figure 4: The DAG for  $a + a * (b - c) + (b - c) * d$  with the value number of every node in red. Arrows go from operator to operand.

| value no. | op          | left | right |
|-----------|-------------|------|-------|
| 1         | id <i>a</i> | —    | —     |
| 2         | id <i>b</i> | —    | —     |
| 3         | id <i>c</i> | —    | —     |
| 4         | —           | 2    | 3     |
| 5         | *           | 1    | 4     |
| 6         | +           | 1    | 5     |
| 7         | id <i>d</i> | —    | —     |
| 8         | *           | 4    | 7     |
| 9         | +           | 6    | 8     |

**left = 1** points to  
record 1 (the leaf *a*)

**right = 4** points to  
record 4 (the node *b - c*)

So record 5 = (\*, 1, 4)  
represents  $a * (b - c)$ .

Figure 5: The array of records after processing the example. Interior records contain *value numbers of their children*; here record 5 refers to records 1 and 4.

**The whole procedure.** Converting a syntax tree into a DAG is a single post-order traversal:

```
int buildDAG(TreeNode t) {           // returns a value number
    if (t is a leaf)
        return findOrCreateLeaf(t); // search (id/num, entry); create if absent
    l = buildDAG(t.left);           // children first (post-order)
    r = buildDAG(t.right);
    return findOrCreateNode(t.op, l, r); // search (op,l,r); create if absent
}
```

The “search” is only a *lookup of an identical record* in the array (or hash table); it is not a second traversal of the tree.

### Looking ahead

Building the tree first and then converting it is easy to understand. Later, after we study syntax-directed definitions (Section 3.8), we will see that the same DAG can be built *directly while parsing*, without constructing the tree at all: the parser simply calls the find-or-create routine each time it recognises an operator.

## 2.3 Three-address code (TAC)

In **three-address code** every instruction has *at most one operator on the right side* and at most three addresses (two operands and one result). Compound expressions are broken into temporaries, so the order of evaluation is made explicit.

For  $a + a * (b - c) + (b - c) * d$  the TAC (from the DAG, reusing  $t_1 = b - c$ ) is:

```
t1 = b - c
t2 = a * t1
t3 = a + t2
t4 = t1 * d
t5 = t3 + t4
```

Without the DAG (from the tree)  $b - c$  would be computed twice.

### 2.3.1 Generating three-address code from a DAG

Three-address code can be generated from the DAG alone; no SDD or SDT is needed.

**Idea.** Every *interior* node of the DAG becomes exactly *one* three-address instruction, with a fresh temporary holding its result. *Leaves* generate no instruction: they only supply a name (a variable such as  $a$ , or a constant). A shared node is still one node, so it produces only one instruction; this is how the DAG removes the common sub-expression.

**Order.** Process the records of the array in increasing order of value number. A node's children were always created before the node, so their value numbers are smaller, and their results are ready when the node is reached. The array order is therefore already a valid evaluation order.

```
for i = 1 to n:                                // records in the array, in index order
  if record i is a leaf:
    name[i] = the variable or constant it stands for
  else:
    // record i = (op, left, right)
    name[i] = newTemp()
    emit( name[i] = name[left] op name[right] )
```

Here  $name[i]$  is the address that holds the value of node  $i$ . For an assignment  $x = \langle \text{expression} \rangle$  we finish with one more instruction,  $x = name[root]$ .

| Record | Content     | $name[left]$ | $name[right]$ | $name[i]$ | Instruction emitted |
|--------|-------------|--------------|---------------|-----------|---------------------|
| 1      | id $a$      | –            | –             | $a$       | (none, leaf)        |
| 2      | id $b$      | –            | –             | $b$       | (none, leaf)        |
| 3      | id $c$      | –            | –             | $c$       | (none, leaf)        |
| 4      | $(-, 2, 3)$ | $b$          | $c$           | $t_1$     | $t_1 = b - c$       |
| 5      | $(*, 1, 4)$ | $a$          | $t_1$         | $t_2$     | $t_2 = a * t_1$     |
| 6      | $(+, 1, 5)$ | $a$          | $t_2$         | $t_3$     | $t_3 = a + t_2$     |
| 7      | id $d$      | –            | –             | $d$       | (none, leaf)        |
| 8      | $(*, 4, 7)$ | $t_1$        | $d$           | $t_4$     | $t_4 = t_1 * d$     |
| 9      | $(+, 6, 8)$ | $t_3$        | $t_4$         | $t_5$     | $t_5 = t_3 + t_4$   |

Table 2: Generating three-address code from the array of Figure ?? (the DAG of  $a + a * (b - c) + (b - c) * d$ ). Node 4 has two parents (nodes 5 and 8) but produces only one instruction; both parents use  $t_1$ .

**Three ways to obtain three-address code.**

| Method                                           | How                                                                                                                                         | Common sub-expressions                                     |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| 1. From a finished DAG                           | Scan the array in index order (above); no SDD/SDT needed.                                                                                   | Computed once (shared nodes).                              |
| 2. During DAG construction                       | Whenever find-or-create <i>creates</i> a node, emit its instruction; when it finds an existing node, emit nothing. Needs SDT (Section 3.8). | Computed once.                                             |
| 3. SDD/SDT directly from the parse (Section 5.1) | Each reduction emits an instruction using <i>newTemp</i> and <i>gen</i> ; no DAG is built.                                                  | Computed again each time they occur (e.g., $b - c$ twice). |

SDD/SDT is the *mechanism* for generating code in one pass while parsing; the DAG is what gives *sharing*. Methods 1 and 2 give the same code.

**Caution: assignments inside the DAG**

If the DAG covers several statements and a variable is *reassigned* in between (e.g.,  $a = a + 1$ ), a later use of  $a$  is a different value and must not share the old leaf or any node built from it. Compilers handle this by creating a new leaf (a new value number) after each assignment. For single expressions, as in these notes, the issue does not arise.

**2.3.2 Addresses and instructions**

An **address** can be: (i) a *name* (source variable; a pointer to its symbol-table entry), (ii) a *constant*, or (iii) a compiler-generated *temporary*.

| Instruction form                            | Meaning                                    |
|---------------------------------------------|--------------------------------------------|
| $x = y \text{ op } z$                       | binary assignment                          |
| $x = \text{op } y$                          | unary assignment (minus, not, conversions) |
| $x = y$                                     | copy                                       |
| goto L                                      | unconditional jump                         |
| if x goto L / ifFalse x goto L              | conditional jump on truth value            |
| if x relop y goto L                         | conditional jump on comparison             |
| param x, call p, n, y = call p, n, return y | procedure calls                            |
| $x = y[i], x[i] = y$                        | indexed copy (memory at address $y + i$ )  |
| $x = \&y, x = *y, *x = y$                   | address and pointer assignments            |
| L:                                          | label (a symbolic name for an instruction) |

Table 3: Common three-address instruction forms.

**2.3.3 Representations of three-address code**

Take the source statement  $a = b * -c + b * -c$ .

**(a) Three-address code**

```

t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a  = t5

```

**(b) Quadruples – (op, arg1, arg2, result)**

|   | op    | arg <sub>1</sub> | arg <sub>2</sub> | result |
|---|-------|------------------|------------------|--------|
| 0 | minus | c                |                  | t1     |
| 1 | *     | b                | t1               | t2     |
| 2 | minus | c                |                  | t3     |
| 3 | *     | b                | t3               | t4     |
| 4 | +     | t2               | t4               | t5     |
| 5 | =     | t5               |                  | a      |

**(c) Triples – (op, arg1, arg2); a result is referred to by the *position* of its triple.**

|   | op    | arg <sub>1</sub> | arg <sub>2</sub> |
|---|-------|------------------|------------------|
| 0 | minus | c                |                  |
| 1 | *     | b                | (0)              |
| 2 | minus | c                |                  |
| 3 | *     | b                | (2)              |
| 4 | +     | (1)              | (3)              |
| 5 | =     | a                | (4)              |

**(d) Indirect triples – a list of pointers to triples.**

| instruction | pointer |
|-------------|---------|
| 35          | (0)     |
| 36          | (1)     |
| 37          | (2)     |
| 38          | (3)     |
| 39          | (4)     |
| 40          | (5)     |

(triples as in (c))

|                  | Advantage                                                                                         | Disadvantage                                                                |
|------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Quadruples       | Results are named explicitly (temporaries); instructions can be moved freely during optimisation. | Needs extra space for temporaries and symbol-table entries.                 |
| Triples          | No temporaries; more compact.                                                                     | Moving a triple changes positions, so all references to it must be updated. |
| Indirect triples | Reordering only rearranges the pointer list, so triples keep their positions.                     | Extra level of indirection; an additional list is needed.                   |

Table 4: Comparison of the three representations.

### 2.3.4 Static single-assignment (SSA) form – a brief note

In **SSA form** every variable is assigned exactly once; different assignments to the same source variable get different subscripts, and a  $\phi$ -function merges values at control-flow joins:

```

p = a + b
q = p - c
p = q * d
p = e - p
q = p + q

```

```

p1 = a + b
q1 = p1 - c
p2 = q1 * d
p3 = e - p2
q2 = p3 + q1

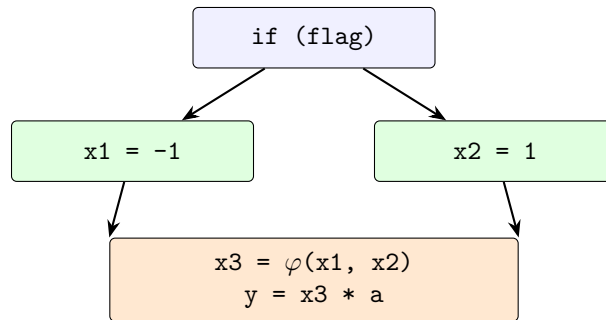
```

```

if (flag) x = -1; else x = 1; y = x * a;
==>  if (flag) x1 = -1; else x2 = 1; x3 = phi(x1, x2); y = x3 * a;

```

The function  $\varphi(x_1, x_2)$  means “take  $x_1$  if control came from the first branch,  $x_2$  if it came from the second”. It is not a real instruction; it marks a join point in the control-flow graph.



### 3 Syntax-Directed Definitions (SDD)

#### 3.1 Why do we need them? Syntax is not enough

So far, parsing has answered only one question: “*Is this string a legal program, and what is its structure?*” A grammar and its parse tree describe **syntax**. They say nothing about **meaning** (*semantics*).

For example, the parse tree of  $3 * 5 + 4$  shows that  $3 * 5$  is grouped first, but nowhere does it contain the number 19. The parse tree of  $a = b * c + d$  does not contain any machine or three-address instructions either. To translate a program we must *attach extra information to the nodes of the parse tree and compute it*. Typical questions are:

- What is the **value** of this expression? (calculators, constant folding)
- What is the **type** of this expression? (type checking)
- In which temporary or variable is its result **stored**? (code generation)
- Which **instructions** implement this statement? (code generation)
- To which **label** should control jump if this condition is true? (control flow)

A **syntax-directed definition** is the formal tool that answers such questions. The idea is simple:

**Grammar + extra information (attributes) on every grammar symbol  
+ small rules (semantic rules) that say how to compute that information  
from the production being used.**

“Syntax-directed” means that the computation is *driven by the syntactic structure* (the productions) of the input.

#### 3.2 Attributes

An **attribute** is any value that we attach to a grammar symbol. It helps to picture the parse tree as a tree of *records*: every node has the grammar symbol as its name plus a few fields (the attributes). If  $X$  is a symbol and  $a$  is an attribute, we write  $X.a$  for the value of  $a$  at a particular node labelled  $X$ . Attributes can be of any kind: numbers, strings, types, pointers to symbol-table entries, or even whole code sequences.

| Attribute         | Meaning                                                               | Typical symbols           |
|-------------------|-----------------------------------------------------------------------|---------------------------|
| $E.val$           | numeric value of the expression                                       | $E, T, F$                 |
| $E.type$          | type of the expression ( <code>int</code> , <code>float</code> , ...) | $E$                       |
| $id.lexeme$       | the actual characters of the identifier (supplied by the lexer)       | <b>id</b>                 |
| $digit.lexval$    | numeric value of a digit token (supplied by the lexer)                | <b>digit</b> , <b>num</b> |
| $E.addr$          | name/temporary holding the result of $E$                              | $E$                       |
| $S.code$          | three-address code for statement $S$                                  | $S, E, B$                 |
| $B.true, B.false$ | labels to jump to if $B$ is true or false                             | $B$                       |
| $E.node$          | pointer to the syntax-tree node for $E$                               | $E, T$                    |

Different *occurrences* of the same nonterminal in a production are distinguished by subscripts, e.g., in  $E \rightarrow E_1 + T$  the  $E$  on the left is the *head* and  $E_1$  is the  $E$  in the body; both have a *val*.

### 3.3 Semantic rules: a first tiny example

A **semantic rule** is attached to a production and says how to compute one attribute from other attributes *of the symbols in that same production*. For the production  $E \rightarrow E_1 + T$  the rule

$$E.val = E_1.val + T.val$$

says: “the value of the whole expression is the value of the left sub-expression plus the value of the term.” Consider the grammar  $E \rightarrow E + T \mid T, T \rightarrow \mathbf{digit}$ , together with the rules

| Production                     | Semantic rule                   |
|--------------------------------|---------------------------------|
| $E \rightarrow E_1 + T$        | $E.val = E_1.val + T.val$       |
| $E \rightarrow T$              | $E.val = T.val$                 |
| $T \rightarrow \mathbf{digit}$ | $T.val = \mathbf{digit.lexval}$ |

Evaluate it on the input  $2 + 3$ . The parse tree is  $E \Rightarrow E_1 + T$  with  $E_1 \Rightarrow T \Rightarrow \mathbf{digit}(2)$  and  $T \Rightarrow \mathbf{digit}(3)$ . Working *from the leaves upward*:

1. The lexer supplies  $\mathbf{digit.lexval} = 2$  and  $3$ .
2.  $T \rightarrow \mathbf{digit}$  gives  $T.val = 2$  (left) and  $T.val = 3$  (right).
3.  $E \rightarrow T$  gives  $E_1.val = 2$ .
4.  $E \rightarrow E_1 + T$  gives  $E.val = 2 + 3 = 5$ .

That is all an SDD does: it decorates the parse tree with attribute values by repeatedly applying the rules.

### 3.4 Formal definition

#### Syntax-Directed Definition (SDD)

A **syntax-directed definition** is a context-free grammar together with **attributes** attached to grammar symbols and **semantic rules** attached to productions. For a production  $A \rightarrow \alpha$  every rule has the form

$$b = f(c_1, c_2, \dots, c_k)$$

where  $f$  is a function and either

- (1)  $b$  is a **synthesized** attribute of  $A$  and  $c_1, \dots, c_k$  are attributes of the symbols in  $\alpha$  (the children) or of  $A$  itself; or
- (2)  $b$  is an **inherited** attribute of a symbol  $B$  in  $\alpha$  and  $c_1, \dots, c_k$  are attributes of  $A$  (the parent) or of symbols in  $\alpha$  (the siblings), or of  $B$  itself.

### 3.5 Synthesized and inherited attributes

#### Analogy

Think of the parse tree as a company. A **synthesized** attribute is a *report sent up* to the manager: the manager's value is built from the reports of the subordinates (e.g., the value of  $2 + 3$  is built from the values of 2 and 3). An **inherited** attribute is an *instruction or information passed down or across*, from a parent or from siblings (e.g., the declared type `float` is passed down to every identifier in `float a, b, c`).

- A **synthesized attribute** at a parse-tree node  $N$  is computed from attributes of  $N$ 's *children* (and  $N$  itself). Information flows *bottom-up*. Terminals have synthesized attributes only (supplied by the lexer, e.g., `digit.lexval`).
- An **inherited attribute** at  $N$  is computed from attributes of  $N$ 's *parent and siblings*. Information flows *top-down / sideways*. Useful when the structure of the parse tree does not match the abstract syntax, e.g., passing a type to a list of identifiers, or passing the label to jump to.

#### Example: synthesized attributes only (desk calculator)

| Production                   | Semantic rules                 |
|------------------------------|--------------------------------|
| $L \rightarrow E \text{ n}$  | $L.val = E.val$                |
| $E \rightarrow E_1 + T$      | $E.val = E_1.val + T.val$      |
| $E \rightarrow T$            | $E.val = T.val$                |
| $T \rightarrow T_1 * F$      | $T.val = T_1.val \times F.val$ |
| $T \rightarrow F$            | $T.val = F.val$                |
| $F \rightarrow ( E )$        | $F.val = E.val$                |
| $F \rightarrow \text{digit}$ | $F.val = \text{digit.lexval}$  |

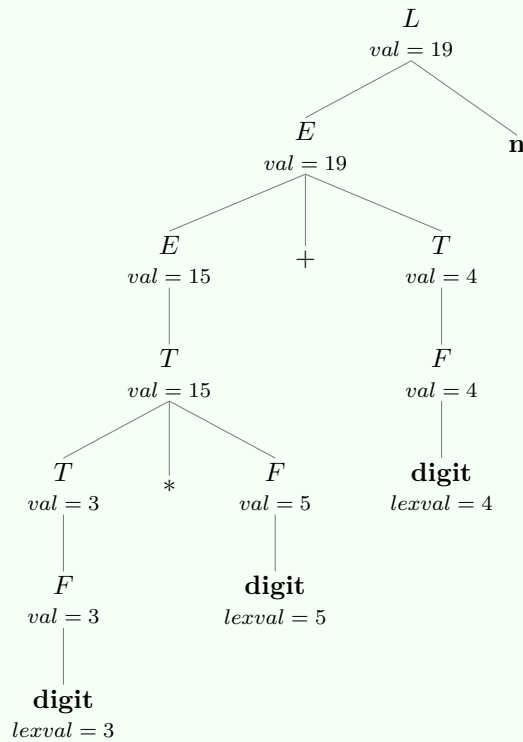


Figure 6: Annotated parse tree for  $3*5+4$  n. Each node shows the value of its synthesized attribute *val*.

#### How to read the figure (evaluation recipe).

1. The lexer supplies the *lexval* of every **digit** leaf (3, 5, 4).
2. Each  $F \rightarrow \mathbf{digit}$  copies it into  $F.val$ ; each  $T \rightarrow F$  copies it up again.
3.  $T \rightarrow T_1 * F$  is applied once both its children are known:  $3 \times 5 = 15$ .
4.  $E \rightarrow E_1 + T$  then adds  $15 + 4 = 19$ , and  $L \rightarrow E \mathbf{n}$  passes 19 to the root.

Every node is computed *after* its children, i.e., in **post-order**. This is natural for synthesized attributes, and it is exactly the order in which an LR parser performs reductions.

### 3.6 Annotated parse trees and dependency graphs

An **annotated parse tree** shows the attribute values at every node. To evaluate attributes we build a **dependency graph**: one node per attribute instance, and an edge  $x \rightarrow y$  whenever  $y$  is computed from  $x$ . A valid evaluation order is any *topological order* of this graph; if the graph has a cycle, no order exists.

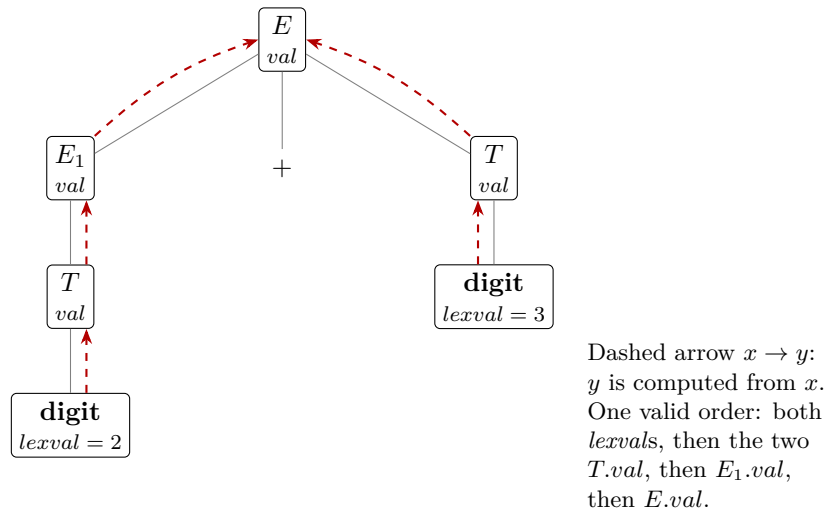


Figure 7: Dependency graph for  $2 + 3$  under the SDD of this section (every attribute is synthesized, so every arrow points *up*).

### A circular definition

For  $A \rightarrow B$  with the rules  $A.s = B.i + 1$  and  $B.i = A.s \times 2$ , the value of  $A.s$  needs  $B.i$ , which needs  $A.s$ . The dependency graph has a *cycle*, so no evaluation order exists. A good SDD must be *non-circular*; the classes S-attributed and L-attributed below guarantee this automatically.

### Side effects

A semantic rule may also do something besides computing a value, e.g., *addType(entry, t)* writes into the symbol table, *print* writes to the output, and *gen* emits an instruction. Such rules are called **side effects**; they are fine as long as the order in which they run does not change the result (or the order is controlled, which is the job of an SDT).

### Example: inherited attributes (declaration list)

| Production                     | Semantic rules                                            |
|--------------------------------|-----------------------------------------------------------|
| $D \rightarrow T L$            | $L.inh = T.type$                                          |
| $T \rightarrow \text{int}$     | $T.type = integer$                                        |
| $T \rightarrow \text{float}$   | $T.type = float$                                          |
| $L \rightarrow L_1, \text{id}$ | $L_1.inh = L.inh; \text{addType}(\text{id.entry}, L.inh)$ |
| $L \rightarrow \text{id}$      | $\text{addType}(\text{id.entry}, L.inh)$                  |

For the input `float id1, id2` the dashed arrows in Fig. 6 are the dependency-graph edges. The type is passed *down and across* from  $T$  to  $L$ , then down through the list to each identifier.

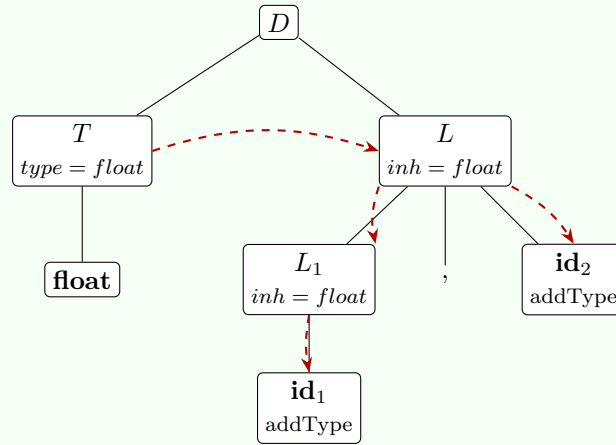


Figure 8: Parse tree for `float id1, id2` with dependency edges (dashed) for the inherited attribute *inh*.

**One valid evaluation order** (any topological order of the dependency graph works):

- |                                    |                                                      |
|------------------------------------|------------------------------------------------------|
| (1) $T.type = float$               | ( $T \rightarrow \text{float}$ ; depends on nothing) |
| (2) $L.inh = T.type$               | ( $D \rightarrow T L$ )                              |
| (3) $L_1.inh = L.inh$              | ( $L \rightarrow L_1, id_2$ )                        |
| (4) $addType(id_2.entry, L.inh)$   | (the identifier after the comma)                     |
| (5) $addType(id_1.entry, L_1.inh)$ | ( $L_1 \rightarrow id_1$ )                           |

Steps (4) and (5) do not depend on each other, so they may be swapped; both identifiers end up with type *float* either way.

### 3.7 Classes of SDDs that are easy to implement

#### S-attributed and L-attributed definitions

- An SDD is **S-attributed** if *every attribute is synthesized*. It can be evaluated in any bottom-up order, e.g., during LR parsing, at the time of reduction.
- An SDD is **L-attributed** if each attribute is either synthesized, or inherited such that, for a production  $A \rightarrow X_1 X_2 \cdots X_n$ , the inherited attribute of  $X_i$  depends only on:
  - (a) inherited attributes of  $A$ , and
  - (b) attributes (inherited or synthesized) of  $X_1, \dots, X_{i-1}$  (the symbols to its *left*), and
  - (c) attributes of  $X_i$  itself, provided there is no cycle.

Evaluation is possible in a single left-to-right depth-first traversal (e.g., during top-down/LL parsing).

Every S-attributed SDD is also L-attributed.

#### Not L-attributed

For  $A \rightarrow B C$ , the rule  $C.i = B.s$  is fine (the inherited attribute of  $C$  uses its *left* sibling  $B$ ). But the rule  $B.i = f(C.x)$  makes the inherited attribute of  $B$  depend on its *right* sibling  $C$ , so the SDD is *not* L-attributed.

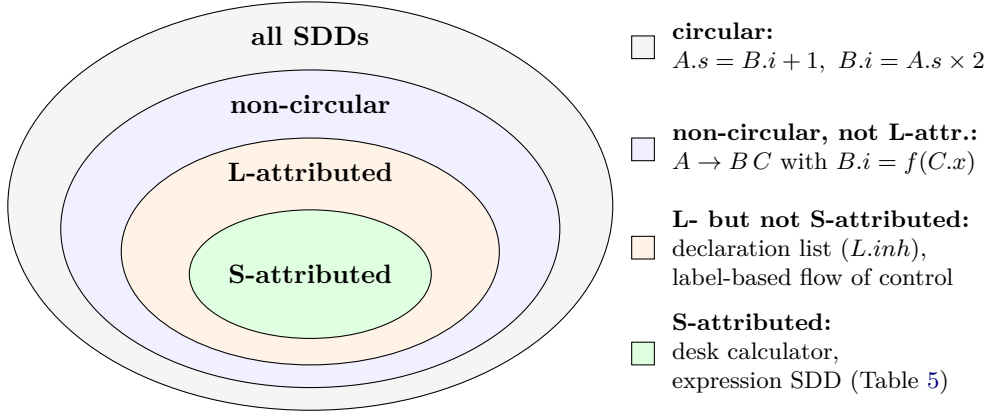


Figure 9: How the classes of SDDs nest, with an example of each.

### 3.8 SDD for constructing syntax trees

Syntax trees can be built using synthesized attribute *node*, which points to the root of a (sub)tree, and constructor functions  $Node(op, c_1, \dots)$  and  $Leaf(id, entry) / Leaf(num, val)$ .

| Production                   | Semantic rules                                               |
|------------------------------|--------------------------------------------------------------|
| $E \rightarrow E_1 + T$      | $E.node = \mathbf{new} Node('+', E_1.node, T.node)$          |
| $E \rightarrow E_1 - T$      | $E.node = \mathbf{new} Node('-', E_1.node, T.node)$          |
| $E \rightarrow T$            | $E.node = T.node$                                            |
| $T \rightarrow ( E )$        | $T.node = E.node$                                            |
| $T \rightarrow \mathbf{id}$  | $T.node = \mathbf{new} Leaf(\mathbf{id}, \mathbf{id}.entry)$ |
| $T \rightarrow \mathbf{num}$ | $T.node = \mathbf{new} Leaf(\mathbf{num}, \mathbf{num}.val)$ |

**Building a DAG directly.** If each  $\mathbf{new} Node(\dots) / \mathbf{new} Leaf(\dots)$  is replaced by the find-or-create routine of Section 2.2.1 (which returns an existing node when an identical one exists), the same SDD builds a DAG during parsing, and  $E.node$  holds a value number.

This is S-attributed. If the grammar is made LL by eliminating left recursion, the same tree is built with an inherited attribute (see the L-attributed SDT in Section 4.6).

## 4 Syntax-Directed Translation Schemes (SDT)

### 4.1 From specification to implementation

An SDD tells us *what* every attribute must equal, but it does not say *when* each rule should run. A real compiler, however, runs the rules *while it is parsing*, in one definite order, and often without ever building the whole parse tree. A **syntax-directed translation scheme (SDT)** solves this: it takes the program fragments of the rules and writes them *inside the production body, at the exact position where they should run*.

The same rule written both ways:

| SDD (specification)                                        | SDT (implementation)                                        |
|------------------------------------------------------------|-------------------------------------------------------------|
| $E \rightarrow E_1 + T$<br>rule: $E.val = E_1.val + T.val$ | $E \rightarrow E_1 + T \quad \{ E.val = E_1.val + T.val \}$ |

|                     | SDD                                                     | SDT                                                                     |
|---------------------|---------------------------------------------------------|-------------------------------------------------------------------------|
| Nature              | High-level specification: rules attached to productions | Concrete implementation: actions embedded in productions                |
| Order of evaluation | Not specified (any order consistent with dependencies)  | Explicit: actions run in the left-to-right order of the production body |
| Rules/actions       | Written next to the production                          | Written in braces $\{\dots\}$ inside the body                           |
| Used for            | Designing and reasoning about the translation           | Building the actual translator                                          |

## 4.2 Definition and the meaning of an action's position

### Syntax-Directed Translation Scheme

A **syntax-directed translation scheme** (SDT) is a context-free grammar with **program fragments** (*semantic actions*) embedded within the production bodies, written in braces. The position of an action defines *when* it is executed during parsing.

$$B \rightarrow X \{a\} Y$$

Here action  $a$  is executed after  $X$  has been recognised (and before  $Y$ ).

**How an SDT is executed.** Imagine the parse tree in which every action appears as an *extra child*, exactly at the position where it is written in the production. Perform a **depth-first traversal from left to right**; whenever the traversal reaches an action, it is executed. Figure 7 shows this for a scheme that translates infix to postfix:  $\text{expr} \rightarrow \text{expr} + \text{term} \{\text{print}('++')\} \mid \text{expr} - \text{term} \{\text{print}('--')\} \mid \text{term}$  and  $\text{term} \rightarrow \text{digit} \{\text{print}(\text{digit})\}$ .

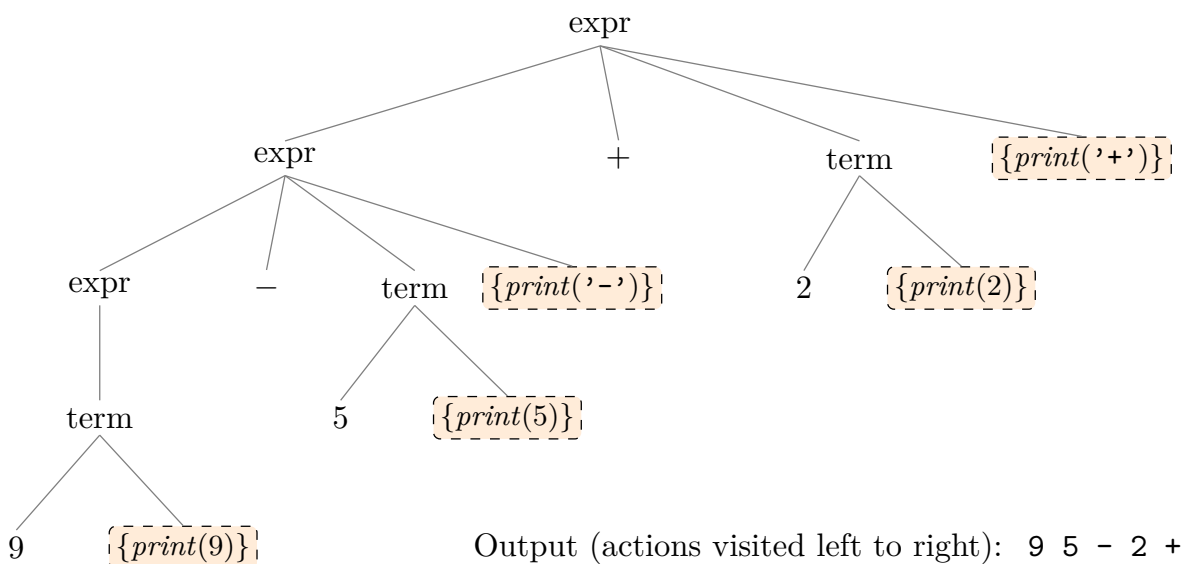


Figure 10: Parse tree of  $9-5+2$  with actions (dashed boxes) as extra children. Visiting the tree depth-first, left to right, executes the actions in the order 9, 5,  $-$ , 2,  $+$ , producing the postfix form.

In practice the parser does not need to build this tree. A top-down parser executes an action at the corresponding point of its recursive-descent procedure. A bottom-up (LR) parser can only act

when it *reduces*: an action at the right end of a body runs at that reduction (a postfix SDT, next subsection), while an action in the middle of a body needs a marker nonterminal (Section ??).

### 4.3 Postfix translation schemes

If the grammar is LR-parsable and the SDD is **S-attributed**, we can build an SDT with every action at the *right end* of its production (a **postfix SDT**). The actions then execute at *reduction time*, so the attribute of each grammar symbol can be stored beside it on the parser stack.

#### Postfix SDT for the desk calculator

|                                |                                       |
|--------------------------------|---------------------------------------|
| $L \rightarrow E \mathbf{n}$   | $\{ \text{print}(E.val) \}$           |
| $E \rightarrow E_1 + T$        | $\{ E.val = E_1.val + T.val \}$       |
| $E \rightarrow T$              | $\{ E.val = T.val \}$                 |
| $T \rightarrow T_1 * F$        | $\{ T.val = T_1.val \times F.val \}$  |
| $T \rightarrow F$              | $\{ T.val = F.val \}$                 |
| $F \rightarrow ( E )$          | $\{ F.val = E.val \}$                 |
| $F \rightarrow \mathbf{digit}$ | $\{ F.val = \mathbf{digit}.lexval \}$ |

**Implementation on the LR parse stack.** Each stack entry holds a grammar symbol and its attribute value. When  $A \rightarrow XYZ$  is reduced, the actions read the values in the top three entries (`stack[top-2].val`, `stack[top-1].val`, `stack[top].val`), compute  $A.val$ , pop three entries, and push  $A$  with the new value.

| Stack (symbol:val)          | Remaining input | Action                                                          |
|-----------------------------|-----------------|-----------------------------------------------------------------|
| \$                          | 3 * 5 + 4 n     | shift                                                           |
| \$ <b>digit</b> :3          | * 5 + 4 n       | reduce $F \rightarrow \mathbf{digit}$                           |
| \$ $F$ :3                   | * 5 + 4 n       | reduce $T \rightarrow F$                                        |
| \$ $T$ :3                   | * 5 + 4 n       | shift *, shift <b>digit</b>                                     |
| \$ $T$ :3 * <b>digit</b> :5 | + 4 n           | reduce $F \rightarrow \mathbf{digit}$                           |
| \$ $T$ :3 * $F$ :5          | + 4 n           | reduce $T \rightarrow T * F$ ( $3 \times 5 = 15$ )              |
| \$ $T$ :15                  | + 4 n           | reduce $E \rightarrow T$                                        |
| \$ $E$ :15                  | + 4 n           | shift +, shift <b>digit</b> , reduce to $T$ :4                  |
| \$ $E$ :15 + $T$ :4         | n               | reduce $E \rightarrow E + T$ ( $15 + 4 = 19$ )                  |
| \$ $E$ :19                  | n               | shift <b>n</b> , reduce $L \rightarrow E \mathbf{n}$ , print 19 |

### 4.4 SDTs with actions inside productions

When an action is in the middle of a production, it is performed *immediately after all grammar symbols to its left have been matched*. A top-down parser does exactly that: the action is just a statement placed at that point of the recursive-descent procedure. An LR parser cannot: it only takes decisions at reductions, so a mid-production action must first be turned into a reduction using a marker nonterminal (Section ??).

| Where the action is | Top-down (LL / recursive descent)                | Bottom-up (LR)                                                        |
|---------------------|--------------------------------------------------|-----------------------------------------------------------------------|
| At the right end    | run after the last symbol of the body is matched | run at the reduction of the body                                      |
| In the middle       | run at that point of the procedure               | not possible directly: introduce a marker $M \rightarrow \varepsilon$ |

#### Example: infix to postfix (action placed after the operand)

$$\begin{aligned}
 E &\rightarrow T R \\
 R &\rightarrow + T \{ \text{print}(' + ') \} R_1 \mid - T \{ \text{print}(' - ') \} R_1 \mid \varepsilon \\
 T &\rightarrow \text{num} \{ \text{print}(\text{num.val}) \}
 \end{aligned}$$

For 9-5+2 the output is 95-2+.

### 4.5 Marker nonterminals (actions in the middle, for bottom-up parsing)

Suppose an action must appear in the middle of a production, e.g.,  $B \rightarrow X \{a\} Y$ . For an LR parser we replace the action by a **marker nonterminal**  $M$  with an empty production:

$$B \rightarrow X M Y, \quad M \rightarrow \varepsilon \{a\}$$

The action  $a$  is then executed upon reducing  $M \rightarrow \varepsilon$ , which happens exactly when  $X$  is on top of the stack and  $Y$  has not been read yet.

#### Example: the infix-to-postfix scheme under LR parsing

The top-down scheme  $R \rightarrow + T \{ \text{print}(' + ') \} R_1$  has its action in the middle. For an LR parser we write

$$R \rightarrow + T M R_1, \quad M \rightarrow \varepsilon \{ \text{print}(' + ') \}$$

When  $M \rightarrow \varepsilon$  is reduced, the operand  $T$  has just been reduced (and printed), so the output order is unchanged: 9 5 - 2 + for 9-5+2.

#### Markers can create conflicts

A marker forces the parser to decide to reduce  $M \rightarrow \varepsilon$  *before* it knows which production will finally be used. If two productions start with the same symbols, say  $S \rightarrow X M Y \mid X Z$ , the parser must choose, using only the next input token, whether to reduce  $M$ . If  $Y$  and  $Z$  can start with the same token, the result is a conflict and the grammar is no longer LR(1). The markers in backpatching ( $M$  before a statement,  $N$  before **else**) are safe because they always occur at a point where the parser can already tell what is coming.

We will use markers  $M$  and  $N$  extensively in backpatching (Section 8).

### 4.6 Eliminating left recursion from an SDT

A top-down parser cannot use a left-recursive grammar such as  $E \rightarrow E_1 + T$ , but the *translation* we want (a left-associative tree or value) is naturally defined by that grammar. The fix is to rewrite the grammar *and* the actions together, using an inherited attribute that carries the result built so far.

## General transformation

$$A \rightarrow A_1 Y \{ A.a = g(A_1.a, Y.y) \} \mid X \{ A.a = f(X.x) \}$$

becomes

$$A \rightarrow X \{ R.i = f(X.x) \} R \{ A.a = R.s \}$$

$$R \rightarrow Y \{ R_1.i = g(R.i, Y.y) \} R_1 \{ R.s = R_1.s \} \mid \varepsilon \{ R.s = R.i \}$$

$R.i$  (inherited) carries the value accumulated *so far*;  $R.s$  (synthesized) carries the final result back to the top.

Example: evaluating  $9 - 5 + 2$  with a left-recursion-free grammar

The left-recursive grammar  $E \rightarrow E_1 + T \mid E_1 - T \mid T, T \rightarrow \text{num}$ , with  $E.val = E_1.val \pm T.val$ , becomes

$$E \rightarrow T \{ R.i = T.val \} R \{ E.val = R.s \}$$

$$R \rightarrow + T \{ R_1.i = R.i + T.val \} R_1 \{ R.s = R_1.s \}$$

$$\mid - T \{ R_1.i = R.i - T.val \} R_1 \{ R.s = R_1.s \} \mid \varepsilon \{ R.s = R.i \}$$

$$T \rightarrow \text{num} \{ T.val = \text{num.lexval} \}$$

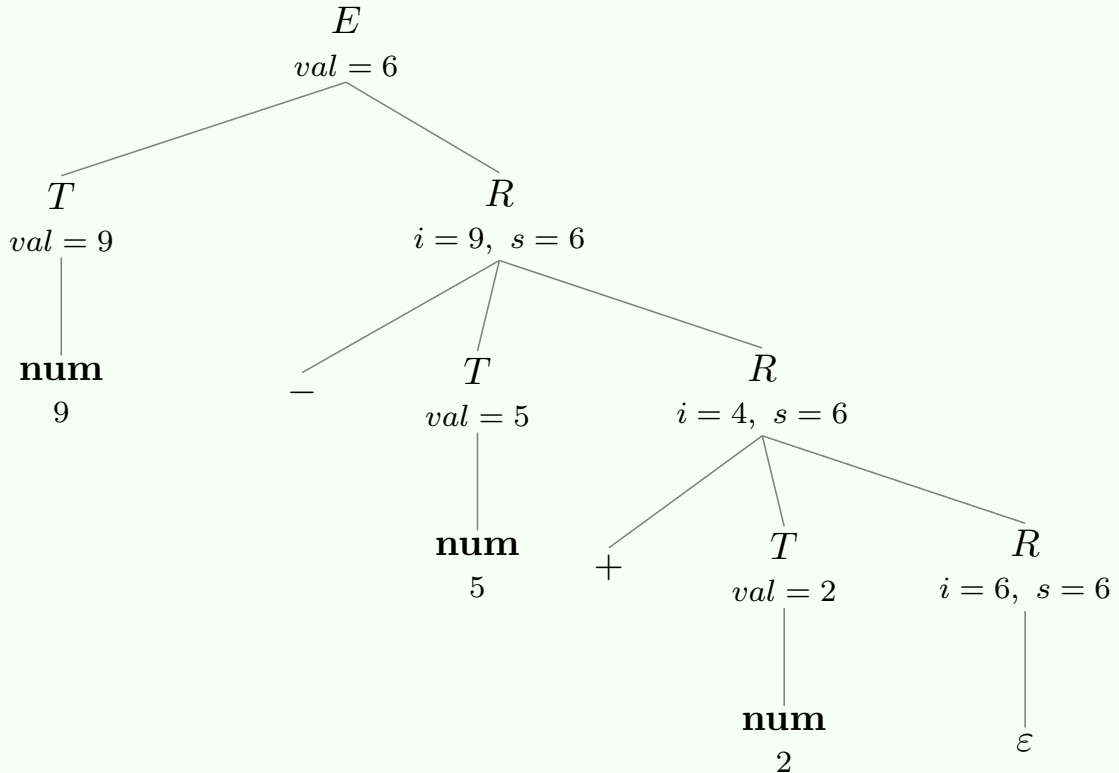


Figure 11: Annotated parse tree for  $9-5+2$ . The inherited  $R.i$  flows down and to the right ( $9 \rightarrow 4 \rightarrow 6$ : first  $9 - 5$ , then  $4 + 2$ ); the synthesized  $R.s$  then carries the final 6 back up to  $E.val$ .

The tree leans to the right, but the arithmetic is still left-associative:  $(9 - 5) + 2 = 6$ , not  $9 - (5 + 2) = 2$ .

## 4.7 Converting an L-attributed SDD into an SDT

### Rules for converting an L-attributed SDD to an SDT

1. **Inherited attribute** of a nonterminal  $X$  in the body: embed the action that computes it *immediately before*  $X$  in the body. If several inherited attributes of  $X$  depend on one another, order the actions appropriately.
2. **Synthesized attribute** of the head  $A$ : place the action that computes it at the *end* of the production body.

### Example: syntax tree for a left-recursion-free grammar

After removing left recursion from  $E \rightarrow E_1 + T \mid E_1 - T \mid T$ :

$$\begin{aligned}
 E &\rightarrow T \{ R.i = T.node \} R \{ E.node = R.s \} \\
 R &\rightarrow + T \{ R_1.i = \mathbf{new} \text{ Node}('+', R.i, T.node) \} R_1 \{ R.s = R_1.s \} \\
 R &\rightarrow - T \{ R_1.i = \mathbf{new} \text{ Node}('-', R.i, T.node) \} R_1 \{ R.s = R_1.s \} \\
 R &\rightarrow \varepsilon \{ R.s = R.i \} \\
 T &\rightarrow \mathbf{id} \{ T.node = \mathbf{new} \text{ Leaf}(\mathbf{id}, \mathbf{id.entry}) \} \\
 T &\rightarrow \mathbf{num} \{ T.node = \mathbf{new} \text{ Leaf}(\mathbf{num}, \mathbf{num.val}) \}
 \end{aligned}$$

Here  $R.i$  (inherited) carries the tree built *so far*;  $R.s$  (synthesized) returns the final tree. For  $a - 4 + c$  the tree  $((a - 4) + c)$  is produced with the correct left-associativity.

## 5 Translation of Expressions and Assignments

### 5.1 SDD for three-address code

We use the following:

- $E.addr$ : the address (name, constant, or temporary) that holds the value of  $E$ .
- $E.code$  (or  $S.code$ ): the three-address instructions that evaluate  $E$  (or execute  $S$ ).
- $newTemp()$  (written  $\mathbf{new Temp}$ ): returns a fresh temporary  $t_1, t_2, \dots$  in successive calls.
- $\mathbf{gen}(\dots)$ : builds a three-address instruction.  $\parallel$  denotes concatenation of instruction sequences.

#### Remark

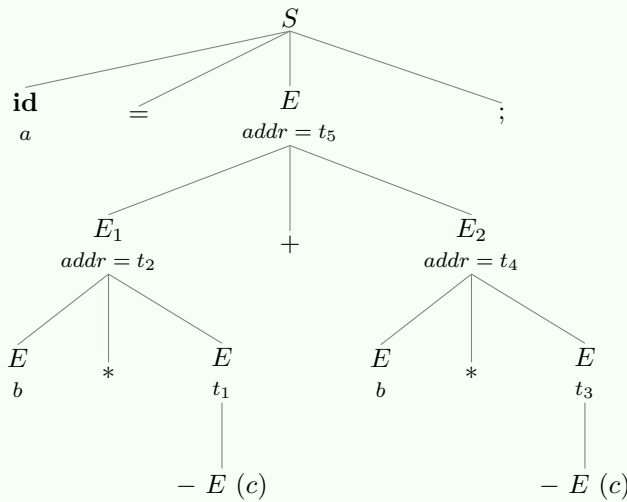
Here  $top$  is the current symbol table;  $top.get(name)$  returns the symbol-table entry. The code of a node is the *concatenation* of its children's code followed by one new instruction, so the instructions are produced in postfix (evaluation) order. The  $.code$  attribute is a long string, so this SDD is inefficient in practice, which motivates the incremental approach below.

The grammar  $E \rightarrow E_1 + E_2 \mid E_1 * E_2 \mid -E_1 \mid (E_1) \mid \mathbf{id}$  is ambiguous. Precedence ( $-$  highest, then  $*$ , then  $+$ ) and left-associativity are supplied either by parser-generator declarations or by rewriting the grammar with levels  $E, T, F$ . This does not change the semantic rules: they are written once per *operator*, whatever the grammar levels are called. The  $*$  rule is the  $+$  rule with the operator replaced; the rule for binary  $-$  and for  $/$  is identical again.

| Production                      | Semantic rules                                                                                                            |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| $S \rightarrow \text{id} = E ;$ | $S.code = E.code \parallel \text{gen}(top.get(\text{id.lexeme}) = E.addr)$                                                |
| $E \rightarrow E_1 + E_2$       | $E.addr = \text{new Temp}()$<br>$E.code = E_1.code \parallel E_2.code \parallel \text{gen}(E.addr = E_1.addr + E_2.addr)$ |
| $E \rightarrow E_1 * E_2$       | $E.addr = \text{new Temp}()$<br>$E.code = E_1.code \parallel E_2.code \parallel \text{gen}(E.addr = E_1.addr * E_2.addr)$ |
| $E \rightarrow - E_1$           | $E.addr = \text{new Temp}()$<br>$E.code = E_1.code \parallel \text{gen}(E.addr = \text{minus } E_1.addr)$                 |
| $E \rightarrow ( E_1 )$         | $E.addr = E_1.addr; \quad E.code = E_1.code$                                                                              |
| $E \rightarrow \text{id}$       | $E.addr = top.get(\text{id.lexeme}); \quad E.code = ''$                                                                   |

Table 5: SDD to produce three-address code for expressions and assignments (Dragon Book Fig. 6.19).

**Worked example:**  $a = b * -c + b * -c ;$



Concatenating the code in the order of evaluation gives:

```

t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5

```

## 5.2 Incremental translation (SDT with `gen` emitting directly)

Building  $E.code$  as a string copies code repeatedly. In an *incremental* scheme,  $gen()$  **appends** the instruction to an output stream/array as soon as the action is executed. The code attribute is not stored. Since actions run in the correct order (postfix), the instructions come out in the right order.

**Worked example (incremental SDT):**  $x = a * b + c * d ;$

An LR parser reduces from left to right, and every reduction that creates a temporary emits *one* instruction at that moment. Reductions  $E \rightarrow \text{id}$  only set  $addr$  and emit nothing. The

| Production                      | Translation scheme (actions in braces)                                                                    |
|---------------------------------|-----------------------------------------------------------------------------------------------------------|
| $S \rightarrow \text{id} = E ;$ | $\{ \text{gen}(\text{top.get}(\text{id.lexeme}) = E.\text{addr}); \}$                                     |
| $E \rightarrow E_1 + E_2$       | $\{ E.\text{addr} = \text{new Temp}(); \text{gen}(E.\text{addr} = E_1.\text{addr} + E_2.\text{addr}); \}$ |
| $  E_1 * E_2$                   | $\{ E.\text{addr} = \text{new Temp}(); \text{gen}(E.\text{addr} = E_1.\text{addr} * E_2.\text{addr}); \}$ |
| $  -E_1$                        | $\{ E.\text{addr} = \text{new Temp}(); \text{gen}(E.\text{addr} = \text{minus } E_1.\text{addr}); \}$     |
| $  (E_1)$                       | $\{ E.\text{addr} = E_1.\text{addr}; \}$                                                                  |
| $  \text{id}$                   | $\{ E.\text{addr} = \text{top.get}(\text{id.lexeme}); \}$                                                 |

Table 6: Incremental SDT for expressions (all actions are at the right end, so it is a postfix SDT, suitable for LR parsing).

red numbers show the order in which instructions are emitted.

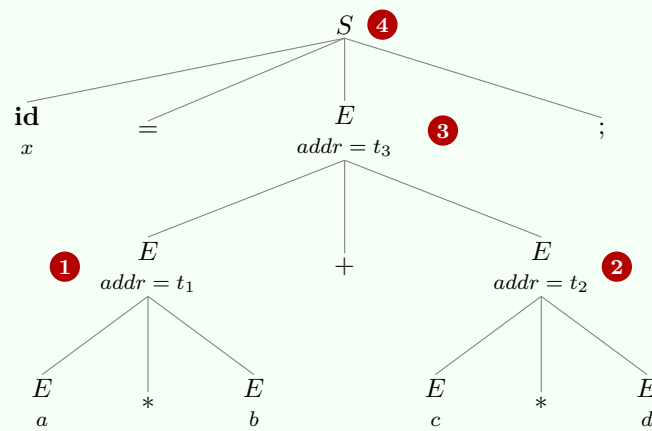


Figure 12: Parse tree of  $x = a * b + c * d ;$  with the order of emission.

| # | Reduction                             | Instruction emitted |
|---|---------------------------------------|---------------------|
| 1 | $E \rightarrow E_1 * E_2$ ( $a * b$ ) | $t1 = a * b$        |
| 2 | $E \rightarrow E_1 * E_2$ ( $c * d$ ) | $t2 = c * d$        |
| 3 | $E \rightarrow E_1 + E_2$             | $t3 = t1 + t2$      |
| 4 | $S \rightarrow \text{id} = E ;$       | $x = t3$            |

Note that  $c * d$  is reduced only after the parser has seen the  $;$ : precedence decides that  $*$  binds tighter, so the  $+$  cannot be reduced earlier.

### 5.3 Addressing array elements

Let an array be stored in a contiguous block of memory with base address *base* and element width *w*.

**One-dimensional array**  $A[\text{low} \dots \text{high}]$ :

$$\text{address of } A[i] = \text{base} + (i - \text{low}) \times w$$

If  $\text{low} = 0$  this is simply  $\text{base} + i \times w$ .

**Two-dimensional array** (row-major)  $A[n_1][n_2]$  with element width  $w$ :

$$\text{address of } A[i_1][i_2] = \text{base} + (i_1 \times n_2 + i_2) \times w$$

**$k$  dimensions**  $A[n_1] \cdots [n_k]$  (row-major, 0-based):

$$\text{base} + ((\cdots ((i_1 n_2 + i_2) n_3 + i_3) \cdots) n_k + i_k) \times w$$

Equivalently, using the *widths of sub-arrays*  $w_1 = n_2 \cdots n_k \cdot w$ ,  $w_2 = n_3 \cdots n_k \cdot w, \dots$ ,  $w_k = w$ :

$$\text{base} + i_1 \times w_1 + i_2 \times w_2 + \cdots + i_k \times w_k.$$

This form is the one used in the translation scheme.

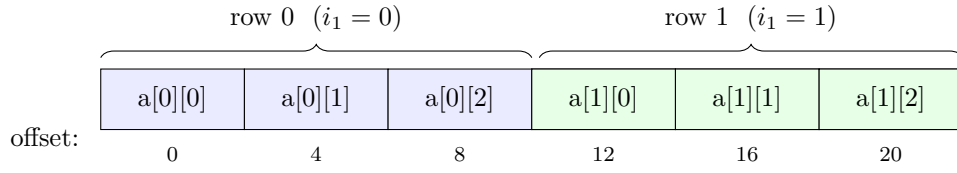


Figure 13: Row-major layout of `int a[2][3]` ( $w = 4$ ). Offset of  $a[i][j]$  is  $i \times 12 + j \times 4$ .

## 5.4 Translation of array references

Grammar (with nonterminal  $L$  for an array location):

$$\begin{aligned} S &\rightarrow \mathbf{id} = E ; \mid L = E ; \\ E &\rightarrow E_1 + E_2 \mid \mathbf{id} \mid L \\ L &\rightarrow \mathbf{id} [ E ] \mid L_1 [ E ] \end{aligned}$$

Attributes of  $L$ :  $L.array$  (symbol-table entry of the array),  $L.type$  (type of the *sub-array* generated by  $L$ ),  $L.addr$  (a temporary accumulating the *offset*).

| Production                        | Semantic action                                                                                                                                                                                 |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $S \rightarrow \mathbf{id} = E ;$ | $\{ \text{gen}(\text{top.get}(\mathbf{id.lexeme}) = E.addr); \}$                                                                                                                                |
| $S \rightarrow L = E ;$           | $\{ \text{gen}(L.array.base [ L.addr ] = E.addr); \}$                                                                                                                                           |
| $E \rightarrow E_1 + E_2$         | $\{ E.addr = \mathbf{new Temp}(); \text{gen}(E.addr = E_1.addr + E_2.addr); \}$                                                                                                                 |
| $E \rightarrow \mathbf{id}$       | $\{ E.addr = \text{top.get}(\mathbf{id.lexeme}); \}$                                                                                                                                            |
| $E \rightarrow L$                 | $\{ E.addr = \mathbf{new Temp}(); \text{gen}(E.addr = L.array.base [ L.addr ]); \}$                                                                                                             |
| $L \rightarrow \mathbf{id} [ E ]$ | $\{ L.array = \text{top.get}(\mathbf{id.lexeme}); L.type = L.array.type.elem;$<br>$L.addr = \mathbf{new Temp}(); \text{gen}(L.addr = E.addr * L.type.width); \}$                                |
| $L \rightarrow L_1 [ E ]$         | $\{ L.array = L_1.array; L.type = L_1.type.elem;$<br>$t = \mathbf{new Temp}(); L.addr = \mathbf{new Temp}();$<br>$\text{gen}(t = E.addr * L.type.width); \text{gen}(L.addr = L_1.addr + t); \}$ |

Table 7: Semantic actions for array references (Dragon Book Fig. 6.22).

**Worked example:  $c + a[i][j]$  where  $a$  is `int a[2][3]`**

Types:  $a.type = array(2, array(3, integer))$ , so the width of one row is  $3 \times 4 = 12$  and of an element is 4.

1.  $L \rightarrow id[E]$  with  $E = i$ :  $L.type = array(3, integer)$ , width 12  $\Rightarrow t1 = i * 12$ .
2.  $L \rightarrow L_1[E]$  with  $E = j$ :  $L.type = integer$ , width 4  $\Rightarrow t2 = j * 4, t3 = t1 + t2$ .
3.  $E \rightarrow L$ :  $t4 = a[t3]$ .
4.  $E \rightarrow E_1 + E_2$ :  $t5 = c + t4$ .

```
t1 = i * 12
t2 = j * 4
t3 = t1 + t2
t4 = a[t3]
t5 = c + t4
```

And for the assignment  $x = a[i][j]$  the code is the first four instructions followed by  $x = t4$ .

**Worked example: a three-dimensional array and a store**

Let `int a[2][3][4]` (so  $w = 4$ ). The widths of the sub-arrays are  $w_3 = 4$  (one element),  $w_2 = 4 \cdot 4 = 16$  (one innermost row of 4 elements) and  $w_1 = 3 \cdot 4 \cdot 4 = 48$  (one  $3 \times 4$  plane); the whole array occupies  $2 \cdot 48 = 96$  bytes. The statement  $x = a[i][j][k]$ ; becomes

```
t1 = i * 48
t2 = j * 16
t3 = t1 + t2
t4 = k * 4
t5 = t3 + t4
t6 = a[t5]
x = t6
```

Each extra subscript adds exactly two instructions (a multiplication and an addition), as in the rule for  $L \rightarrow L_1[E]$ . For a *store*,  $a[i][j] = x$ ; with `int a[2][3]` uses the rule  $S \rightarrow L = E$ ;, which ends with an indexed assignment instead of an indexed load:

```
t1 = i * 12
t2 = j * 4
t3 = t1 + t2
a[t3] = x
```

**Index versus offset**

In  $t4 = a[t3]$  the value in  $t3$  is a *byte offset* from the base of  $a$ , not an element index as in the C source. That is why every subscript is multiplied by the width of the thing it selects. (In the book's three-address code,  $x = y[i]$  means “the value at the memory location  $i$  units beyond  $y$ ”.)

## 6 Type Checking and Type Conversion

**Type expressions** represent the type of a language construct: basic types (`int`, `float`, `char`, `boolean`, `void`),  $array(n, T)$ , *record*, pointer, and function types  $s \rightarrow t$ .

**Type checking** comes in two forms: **synthesis** (the type of an expression is built from the types of its sub-expressions, e.g., for  $E_1 + E_2$ ) and **inference** (the type of a construct is determined from the way it is used).

**Example: three static checks**

Static checking attaches a rule to a production and reports an error when the rule fails. With `int a[10]; int n; float f;;`

| Source                     | Rule that fails                                                                            | Message                                                                       |
|----------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>x = a[2.5];</code>   | in $L \rightarrow \text{id}[E]$ the index $E$ must have type <code>int</code>              | array index must be an integer                                                |
| <code>y = a + n;</code>    | <code>+</code> is defined for numeric types only; $a$ has type <code>array(10, int)</code> | operand of <code>+</code> is not a number                                     |
| <code>if (f) x = 1;</code> | the condition in <code>if</code> ( $E$ ) must have type <code>boolean</code>               | <code>float</code> used as condition (C would accept it and test $f \neq 0$ ) |

Each check is just one more semantic rule: it computes nothing, it only compares attribute values and calls an error routine, which is another example of a rule with a side effect.

**6.1 Type conversion (coercion)**

For  $E \rightarrow E_1 + E_2$  where operand types may differ (e.g., `int` and `float`), the compiler inserts conversions. Conversions that cannot lose information in principle (*widening*) may be inserted by the compiler automatically (*coercion*); the reverse direction (*narrowing*) needs an explicit cast written by the programmer. Figure ?? shows the usual hierarchy. In these notes we use only `int` < `float`.

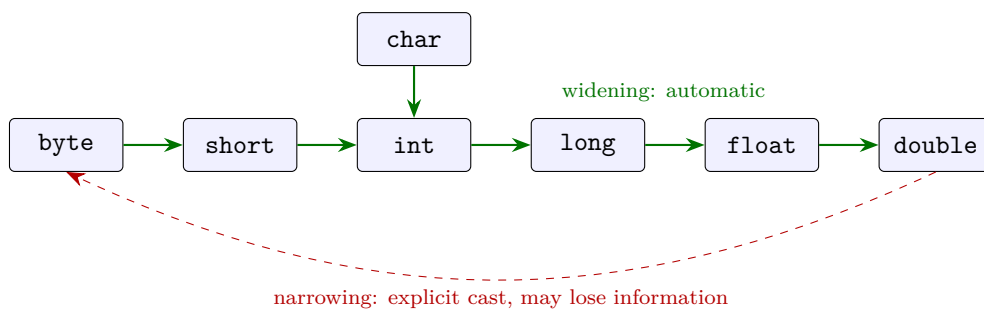


Figure 14: Widening (solid) and narrowing (dashed) conversions between primitive types, Java-style. Converting `long` to `float` is allowed implicitly although a few low-order digits may be rounded.

For  $E \rightarrow E_1 + E_2$  (and likewise for  $-$ ,  $*$ ,  $/$ ) the result has the *wider* of the two operand types,

$$E.type = \max(E_1.type, E_2.type),$$

and the helper `widen( $a, t, w$ )` converts address  $a$  of type  $t$  to type  $w$ :

```

Addr widen(Addr a, Type t, Type w) {
    if (t == w) return a;
    else if (t == integer && w == float) {
        temp = new Temp();
        gen(temp '=' '(float)' a);
        return temp;
    }
    else error;
}

```

| Production                | Semantic action                                                                                                                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $E \rightarrow E_1 + E_2$ | $\{ E.type = \max(E_1.type, E_2.type);$<br>$a_1 = \text{widen}(E_1.addr, E_1.type, E.type);$<br>$a_2 = \text{widen}(E_2.addr, E_2.type, E.type);$<br>$E.addr = \text{new Temp}(); \text{gen}(E.addr = a_1 + a_2); \}$ |

### Example

Let  $x$  be float, and  $i, j$  be int. For  $x = i * j + x$  we get:

```
t1 = i * j          // int * int -> int
t2 = (float) t1     // widen to float
t3 = t2 + x         // float + float
x  = t3
```

### Worked example: widening on *both* sides, $x = i + j * x$

Let  $x$  be float and  $i, j$  be int. The code is emitted at reductions, so the product  $j * x$  (reduced first) comes before the sum:

1.  $E \rightarrow E_1 * E_2$  with  $E_1 = j$  (int) and  $E_2 = x$  (float):  $E.type = \text{float}$ ;  $\text{widen}(j)$  emits  $t1 = (\text{float}) j$ ;  $\text{widen}(x)$  does nothing; then  $t2 = t1 * x$ .
2.  $E \rightarrow E_1 + E_2$  with  $E_1 = i$  (int) and  $E_2$  of type float:  $\text{widen}(i)$  emits  $t3 = (\text{float}) i$ ; then  $t4 = t3 + t2$ .
3.  $S \rightarrow \text{id} = E$ ; emits  $x = t4$ .

```
t1 = (float) j
t2 = t1 * x
t3 = (float) i
t4 = t3 + t2
x  = t4
```

Note that  $i$ , which is written first, is converted *after*  $j$ : the order of the instructions follows the order of the reductions, not the order of the source text.

## 7 Control Flow: Boolean Expressions and Statements

In programming languages, boolean expressions are used for two purposes:

1. to **compute logical values** (e.g., `flag = a < b && c;`), and
2. as **conditional expressions** in `if`, `while`, and `for` (flow-of-control).

We use the grammar

$$B \rightarrow B \parallel B \mid B \&\& B \mid !B \mid (B) \mid E \text{ rel } E \mid \text{true} \mid \text{false}$$

with  $\parallel$  standing for the C operator `||`; `||` and `&&` are left-associative, `||` has lower precedence than `&&`, which is lower than `!`.

### 7.1 Short-circuit (jumping) code

In **short-circuit code**, the operators `&&`, `||` and `!` do not appear in the generated code. Instead a boolean expression is translated into jumps to labels: control *reaches* one position if the expression

is true and another if it is false. In

```
if (x < 100 || x > 200 && x != y) x = 0;
```

the first test  $x < 100$  being true makes the other tests unnecessary. The generated code is:

```

    if x < 100 goto L2
    goto L3
L3:   if x > 200 goto L4
    goto L1
L4:   if x != y goto L2
    goto L1
L2:   x = 0
L1:   ...

```

(Compare with *value* code, which would compute 0/1 into temporaries and use  $\&\&$  and  $\|\|$  at run time.)

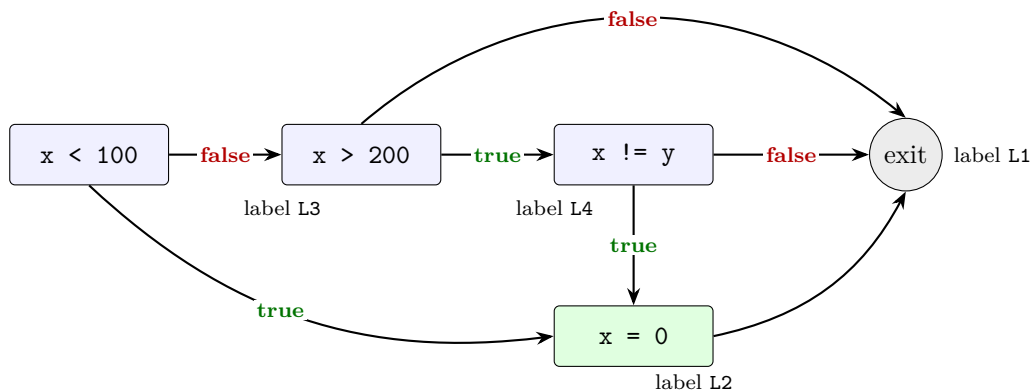


Figure 15: Control-flow view of `if (x < 100 || x > 200 && x != y) x = 0;`. Each test jumps on its own outcome; no boolean value is ever computed. A *true* arrow out of  $x < 100$  skips the other tests.

### Short-circuit evaluation is a semantic guarantee, not only an optimisation

In `if (p != NULL && p->x > 0) ...` the second operand must *not* be evaluated when  $p$  is null, otherwise the program dereferences a null pointer. Jumping code gives this behaviour for free: the test of  $p \rightarrow x$  is only reached through the *true* exit of  $p \neq \text{NULL}$ . Value code that first computes both operands would crash.

**Boolean values in assignments.** A boolean expression is also used to compute a *value*, e.g. `flag = a < b && c;`. The jumping code is reused, and the two exits set the variable:

```

    if a < b goto L1
    goto L3
L1:   if c goto L2
    goto L3
L2:   flag = true
    goto L4
L3:   flag = false
L4:

```

## 7.2 Flow-of-control statements: SDD with inherited labels

Grammar for statements:

$$P \rightarrow S, \quad S \rightarrow \text{assign} \mid \text{if}(B) S_1 \mid \text{if}(B) S_1 \text{ else } S_2 \mid \text{while}(B) S_1 \mid S_1 S_2$$

**Inherited attributes** (labels):

- $B.true$ : the label to jump to if  $B$  is true;  $B.false$ : the label if  $B$  is false.
- $S.next$ : the label of the instruction that follows the code for  $S$ .

**Synthesized attribute:**  $S.code$ ,  $B.code$ . The function  $newlabel()$  creates a fresh label  $L_1, L_2, \dots$ , and  $label(L)$  attaches label  $L$  to the next instruction.

| Production                                         | Semantic rules                                                                                                                                                                                                                                  |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $P \rightarrow S$                                  | $S.next = newlabel(); \quad P.code = S.code \parallel label(S.next)$                                                                                                                                                                            |
| $S \rightarrow \text{assign}$                      | $S.code = \text{assign}.code$                                                                                                                                                                                                                   |
| $S \rightarrow \text{if}(B) S_1$                   | $B.true = newlabel(); \quad B.false = S_1.next = S.next$<br>$S.code = B.code \parallel label(B.true) \parallel S_1.code$                                                                                                                        |
| $S \rightarrow \text{if}(B) S_1 \text{ else } S_2$ | $B.true = newlabel(); \quad B.false = newlabel();$<br>$S_1.next = S_2.next = S.next$<br>$S.code = B.code \parallel label(B.true) \parallel S_1.code \parallel \text{gen}(\text{goto } S.next) \parallel$<br>$label(B.false) \parallel S_2.code$ |
| $S \rightarrow \text{while}(B) S_1$                | $begin = newlabel(); \quad B.true = newlabel(); \quad B.false = S.next;$<br>$S_1.next = begin$<br>$S.code = label(begin) \parallel B.code \parallel label(B.true) \parallel S_1.code \parallel \text{gen}(\text{goto } begin)$                  |
| $S \rightarrow S_1 S_2$                            | $S_1.next = newlabel(); \quad S_2.next = S.next$<br>$S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$                                                                                                                            |

Table 8: SDD for flow-of-control statements (Dragon Book Fig. 6.36).  $S.next$ ,  $B.true$ , and  $B.false$  are inherited.

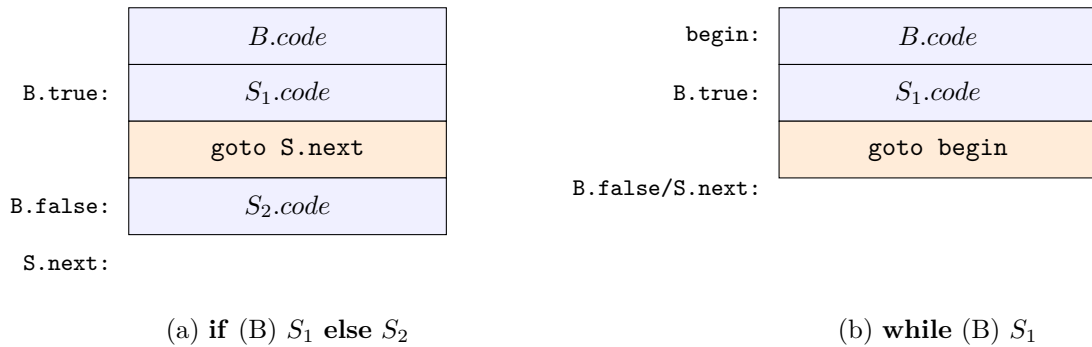


Figure 16: Layout of the generated code.  $B.code$  contains jumps to  $B.true$  or  $B.false$ ; it “falls through” to neither, so the labels can be placed anywhere.

| Production                           | Semantic rules                                                                                                                                                                 |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow B_1 \parallel B_2$    | $B_1.true = B.true; B_1.false = newlabel(); B_2.true = B.true;$<br>$B_2.false = B.false$<br>$B.code = B_1.code \parallel \text{label}(B_1.false) \parallel B_2.code$           |
| $B \rightarrow B_1 \&\& B_2$         | $B_1.true = newlabel(); B_1.false = B.false; B_2.true = B.true;$<br>$B_2.false = B.false$<br>$B.code = B_1.code \parallel \text{label}(B_1.true) \parallel B_2.code$           |
| $B \rightarrow !B_1$                 | $B_1.true = B.false; B_1.false = B.true; B.code = B_1.code$                                                                                                                    |
| $B \rightarrow E_1 \text{ rel } E_2$ | $B.code = E_1.code \parallel E_2.code \parallel$<br>$\text{gen}(\text{if } E_1.addr \text{ rel.op } E_2.addr \text{ goto } B.true) \parallel \text{gen}(\text{goto } B.false)$ |
| $B \rightarrow \text{true}$          | $B.code = \text{gen}(\text{goto } B.true)$                                                                                                                                     |
| $B \rightarrow \text{false}$         | $B.code = \text{gen}(\text{goto } B.false)$                                                                                                                                    |

Table 9: SDD to generate jumping code for boolean expressions (Dragon Book Fig. 6.37).

### 7.3 SDD for boolean expressions (jumping code)

#### How to remember the $\parallel$ and $\&\&$ rules

- $B_1 \parallel B_2$ : if  $B_1$  is true we are done (go to  $B.true$ ); if false, *test*  $B_2$ , so  $B_1.false$  is a new label placed in front of  $B_2.code$ .
- $B_1 \&\& B_2$ : if  $B_1$  is false we are done (go to  $B.false$ ); if true, *test*  $B_2$ , so  $B_1.true$  is a new label in front of  $B_2.code$ .
- $!B_1$ : simply *swap* the true and false labels.

#### Worked example: `if (x < 100 || x > 200 && x != y) x = 0;`

**Step 1.**  $P \rightarrow S$ :  $S.next = L_1$ .

**Step 2.**  $S \rightarrow \text{if}(B)S_1$ :  $B.true = L_2$ ,  $B.false = L_1$ ,  $S_1.next = L_1$ .

**Step 3.**  $B = B_1 \parallel B_2$ , where  $B_1 \equiv x < 100$  and  $B_2 \equiv (x > 200 \&\& x \neq y)$ :

- $B_1$ : true =  $L_2$ , false =  $L_3$  (new).
- $B_2$ : true =  $L_2$ , false =  $L_1$ .

**Step 4.**  $B_2 = B_3 \&\& B_4$  with  $B_3 \equiv x > 200$ ,  $B_4 \equiv x \neq y$ :

- $B_3$ : true =  $L_4$  (new), false =  $L_1$ .
- $B_4$ : true =  $L_2$ , false =  $L_1$ .

**Step 5.** Assemble:  $B_1.code \parallel L_3: B_2.code$ , with  $B_2.code = B_3.code \parallel L_4: B_4.code$ , followed by  $L_2: S_1.code$  and finally  $L_1$ :

```

        if x < 100 goto L2
        goto L3
L3:      if x > 200 goto L4
        goto L1
L4:      if x != y goto L2
        goto L1
L2:      x = 0
L1:
```

Observe the redundant jumps (`goto L3` immediately followed by `L3:`) – a later optimisation phase or the *backpatching* method with fall-through can remove them.

**Worked example: while (a < b) { if (c < d) x = y + z; }**

```

L1:    if a < b goto L2          // begin / B.code
        goto L0                // B.false = S.next
L2:    if c < d goto L3          // inner if: B.true = L3
        goto L1                // inner S.next = begin = L1
L3:    t1 = y + z
        x = t1
        goto L1                // goto begin
L0:                                // S.next

```

**7.4 Avoiding redundant gotos (fall-through)**

Instead of

```

        if x > 200 goto L4
        goto L1
L4:    ...

```

we can generate the shorter form `ifFalse x > 200 goto L1` followed by the code at L4: if the true label is the next instruction (denote it by the special label *fall*), only one jump is needed. The rule for  $B \rightarrow E_1 \text{ rel } E_2$  becomes: let  $test = E_1.addr \text{ rel } op \ E_2.addr$ :

| Condition                                  | Generated jump code              |
|--------------------------------------------|----------------------------------|
| $B.true \neq fall$ and $B.false \neq fall$ | if test goto B.true goto B.false |
| $B.true \neq fall$ and $B.false = fall$    | if test goto B.true              |
| $B.true = fall$ and $B.false \neq fall$    | ifFalse test goto B.false        |
| both = <i>fall</i>                         | (no instruction)                 |

**The statement and operator rules change too.** Let *fall* mean “the next instruction in sequence”. The new rules (Dragon Book Section 6.6.5) are:

| Production                         | Semantic rules                                                                                                                                                                                                                                                                                          |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $S \rightarrow \text{if } (B) S_1$ | $B.true = fall; \ B.false = S_1.next = S.next; \ S.code = B.code \parallel S_1.code$                                                                                                                                                                                                                    |
| $B \rightarrow B_1 \parallel B_2$  | $B_1.true = (B.true \text{ if } \neq fall, \text{ else a new label}); \ B_1.false = fall;$<br>$B_2.true = B.true; \ B_2.false = B.false$<br>$B.code = B_1.code \parallel B_2.code \text{ if } B.true \neq fall; \text{ otherwise}$<br>$B_1.code \parallel B_2.code \parallel \text{label}(B_1.true)$    |
| $B \rightarrow B_1 \ \&\& \ B_2$   | $B_1.true = fall; \ B_1.false = (B.false \text{ if } \neq fall, \text{ else a new label});$<br>$B_2.true = B.true; \ B_2.false = B.false$<br>$B.code = B_1.code \parallel B_2.code \text{ if } B.false \neq fall; \text{ otherwise}$<br>$B_1.code \parallel B_2.code \parallel \text{label}(B_1.false)$ |

The extra label in the “otherwise” case is needed because when  $B_1$  is decisive the control must jump *past*  $B_2$ , to the end of  $B.code$ .

**Worked example: the same statement with fall-through**

For `if (x < 100 || x > 200 && x != y) x = 0`; start with  $B.true = fall$  and  $B.false = L_1$ . For  $B_1 \parallel B_2$ : since  $B.true = fall$ ,  $B_1.true = L_2$  (new) and  $B_1.false = fall$ ;  $B_2$  inherits

$B.true = fall$  and  $B.false = L_1$ . For  $B_2 = B_3 \&\& B_4$ :  $B_3.true = B_4.true = fall$  and  $B_3.false = B_4.false = L_1$ . The relational rule then gives

```

    if x < 100 goto L2
    ifFalse x > 200 goto L1
    ifFalse x != y goto L1
L2:    x = 0
L1:

```

Only 4 instructions (3 tests and the assignment) instead of 7, and no useless **gotos**.

## 8 Backpatching

### 8.1 The problem and the idea

In the one-pass, label-based scheme above we needed *inherited* labels ( $B.true$ ,  $S.next$ ) because a jump may be generated *before* its target is known. Inherited attributes make one-pass **bottom-up** translation awkward. **Backpatching** avoids this:

- When a jump is generated, its *target is left blank*.
- Each such incomplete jump is put on a **list** of jumps that will eventually go to the same target.
- When the target address becomes known, the instructions on the list are *patched* (filled in).

Thus we only need *synthesized* attributes (lists of instruction indices), and the whole translation can be done in a single bottom-up pass.

#### Backpatching primitives

- $makelist(i)$ : create a new list containing only  $i$ , an index into the instruction array; return a pointer to it.
- $merge(p_1, p_2)$ : concatenate the lists pointed to by  $p_1$  and  $p_2$ ; return the merged list.
- $backpatch(p, i)$ : insert  $i$  as the target label for each instruction on the list  $p$ .
- $nextinstr$ : global variable containing the index of the next instruction to be generated.  $emit(\dots)$  appends an instruction and increments it.

### 8.2 Boolean expressions

Synthesized attributes:  $B.truelist$  (instructions that jump when  $B$  is true) and  $B.falselist$  (when false). A **marker nonterminal**  $M \rightarrow \epsilon$  records the index of the next instruction ( $M.instr$ ).

#### Worked example: $x < 100 \parallel x > 200 \&\& x \neq y$ (instructions start at index 100)

**Parse structure:**  $B \Rightarrow B_1 \parallel M_1 B_2$ , where  $B_2 \Rightarrow B_3 \&\& M_2 B_4$ .

1.  $B_1 \equiv x < 100$ : emit 100: `if x<100 goto _`, 101: `goto _`  
 $B_1.truelist = \{100\}$ ,  $B_1.falselist = \{101\}$ .
2.  $M_1.instr = 102$ .
3.  $B_3 \equiv x > 200$ : emit 102: `if x>200 goto _`, 103: `goto _`  
 $B_3.truelist = \{102\}$ ,  $B_3.falselist = \{103\}$ .
4.  $M_2.instr = 104$ .

| Production                           | Semantic action                                                                                                                                                                                                                         |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow B_1 \parallel M B_2$  | $\{ \text{backpatch}(B_1.\text{falselist}, M.\text{instr});$<br>$B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist}); B.\text{falselist} = B_2.\text{falselist}; \}$                                             |
| $B \rightarrow B_1 \&\& M B_2$       | $\{ \text{backpatch}(B_1.\text{truelist}, M.\text{instr});$<br>$B.\text{truelist} = B_2.\text{truelist}; B.\text{falselist} = \text{merge}(B_1.\text{falselist}, B_2.\text{falselist}); \}$                                             |
| $B \rightarrow ! B_1$                | $\{ B.\text{truelist} = B_1.\text{falselist}; B.\text{falselist} = B_1.\text{truelist}; \}$                                                                                                                                             |
| $B \rightarrow ( B_1 )$              | $\{ B.\text{truelist} = B_1.\text{truelist}; B.\text{falselist} = B_1.\text{falselist}; \}$                                                                                                                                             |
| $B \rightarrow E_1 \text{ rel } E_2$ | $\{ B.\text{truelist} = \text{makelist}(\text{nextinstr}); B.\text{falselist} = \text{makelist}(\text{nextinstr} + 1);$<br>$\text{emit}(\text{if } E_1.\text{addr rel.op } E_2.\text{addr goto } \_); \text{emit}(\text{goto } \_); \}$ |
| $B \rightarrow \text{true}$          | $\{ B.\text{truelist} = \text{makelist}(\text{nextinstr}); \text{emit}(\text{goto } \_); \}$                                                                                                                                            |
| $B \rightarrow \text{false}$         | $\{ B.\text{falselist} = \text{makelist}(\text{nextinstr}); \text{emit}(\text{goto } \_); \}$                                                                                                                                           |
| $M \rightarrow \varepsilon$          | $\{ M.\text{instr} = \text{nextinstr}; \}$                                                                                                                                                                                              |

Table 10: Translation scheme for boolean expressions using backpatching (Dragon Book Fig. 6.43).

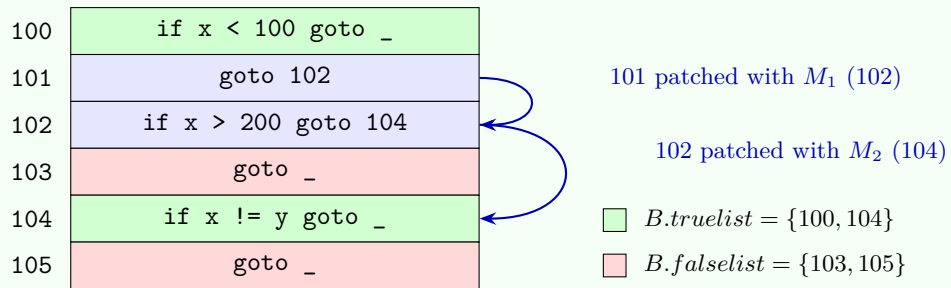
5.  $B_4 \equiv x \neq y$ : emit 104: `if x!=y goto _`, 105: `goto _`  
 $B_4.\text{truelist} = \{104\}$ ,  $B_4.\text{falselist} = \{105\}$ .
6. Reduce  $B_2 \rightarrow B_3 \&\& M_2 B_4$ :  $\text{backpatch}(\{102\}, 104) \Rightarrow$  instruction 102 becomes `if x>200 goto 104`.  
 $B_2.\text{truelist} = \{104\}$ ,  $B_2.\text{falselist} = \text{merge}(\{103\}, \{105\}) = \{103, 105\}$ .
7. Reduce  $B \rightarrow B_1 \parallel M_1 B_2$ :  $\text{backpatch}(\{101\}, 102) \Rightarrow$  instruction 101 becomes `goto 102`.  
 $B.\text{truelist} = \text{merge}(\{100\}, \{104\}) = \{100, 104\}$ ,  $B.\text{falselist} = \{103, 105\}$ .

**Result** (still incomplete – the lists will be patched by the enclosing statement):

```

100:  if x < 100 goto _      <- on truelist
101:  goto 102
102:  if x > 200 goto 104
103:  goto _                <- on falselist
104:  if x != y goto _      <- on truelist
105:  goto _                <- on falselist

```



Blue jumps are already complete. The green and red jumps have a blank target; they are exactly the entries of the two lists that the parent will patch.

### 8.3 Flow-of-control statements

Synthesized attribute  $S.\text{nextlist}$ : list of jumps to the instruction that follows  $S$ . We use markers  $M \rightarrow \varepsilon$  (records  $\text{nextinstr}$ ) and  $N \rightarrow \varepsilon$  (emits an incomplete `goto` and returns it on a list).

Grammar:

$$S \rightarrow \text{if } (B) S_1 \mid \text{if } (B) S_1 \text{ else } S_2 \mid \text{while } (B) S_1 \mid \{ L \} \mid A;$$

$$L \rightarrow L_1 S \mid S$$

| Production                                                          | Semantic action                                                                                                                                                                                                                                                          |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $S \rightarrow \text{if } (B) M S_1$                                | $\{ \text{backpatch}(B.\text{truelist}, M.\text{instr});$<br>$S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist}); \}$                                                                                                                             |
| $S \rightarrow$<br>$\text{if } (B) M_1 S_1 N \text{ else } M_2 S_2$ | $\{ \text{backpatch}(B.\text{truelist}, M_1.\text{instr}); \text{backpatch}(B.\text{falselist}, M_2.\text{instr});$<br>$\text{temp} = \text{merge}(S_1.\text{nextlist}, N.\text{nextlist});$<br>$S.\text{nextlist} = \text{merge}(\text{temp}, S_2.\text{nextlist}); \}$ |
| $S \rightarrow \text{while } M_1 (B) M_2 S_1$                       | $\{ \text{backpatch}(S_1.\text{nextlist}, M_1.\text{instr});$<br>$\text{backpatch}(B.\text{truelist}, M_2.\text{instr});$<br>$S.\text{nextlist} = B.\text{falselist}; \text{emit}(\text{goto } M_1.\text{instr}); \}$                                                    |
| $S \rightarrow \{ L \}$                                             | $\{ S.\text{nextlist} = L.\text{nextlist}; \}$                                                                                                                                                                                                                           |
| $S \rightarrow A;$                                                  | $\{ S.\text{nextlist} = \text{null}; \}$                                                                                                                                                                                                                                 |
| $L \rightarrow L_1 M S$                                             | $\{ \text{backpatch}(L_1.\text{nextlist}, M.\text{instr}); L.\text{nextlist} = S.\text{nextlist}; \}$                                                                                                                                                                    |
| $L \rightarrow S$                                                   | $\{ L.\text{nextlist} = S.\text{nextlist}; \}$                                                                                                                                                                                                                           |
| $M \rightarrow \varepsilon$                                         | $\{ M.\text{instr} = \text{nextinstr}; \}$                                                                                                                                                                                                                               |
| $N \rightarrow \varepsilon$                                         | $\{ N.\text{nextlist} = \text{makelist}(\text{nextinstr}); \text{emit}(\text{goto } \_); \}$                                                                                                                                                                             |

Table 11: Translation of statements with backpatching (Dragon Book Fig. 6.46).

### Why the markers?

*M remembers a position:* “this is where the code for  $S_1$  (or for the test) begins”. *N generates the jump over the else-part* and returns it for later patching. In the **while** rule the loop-back jump uses  $M_1$ , which was recorded *before*  $B.\text{code}$  was generated.

### Worked example: while (a < b) { if (c < d) x = y + z; }

Assume numbering starts at 100;  $M_1.\text{instr} = 100$ .

1.  $B \equiv a < b$ : 100: if a<b goto \_, 101: goto \_;  $B.\text{truelist} = \{100\}$ ,  $B.\text{falselist} = \{101\}$ .
2.  $M_2.\text{instr} = 102$ .
3. Inner  $B' \equiv c < d$ : 102: if c<d goto \_, 103: goto \_;  $B'.\text{truelist} = \{102\}$ ,  $B'.\text{falselist} = \{103\}$ .
4.  $M.\text{instr} = 104$ ; assignment  $x = y + z$ : 104: t1 = y + z, 105: x = t1;  $S_1.\text{nextlist} = \text{null}$ .
5. Inner **if**:  $\text{backpatch}(\{102\}, 104)$ ;  $S.\text{nextlist} = \text{merge}(\{103\}, \text{null}) = \{103\}$ .
6. **while**:  $\text{backpatch}(\{103\}, 100)$  (i.e., 103: goto 100);  $\text{backpatch}(\{100\}, 102)$  (i.e., 100: if a<b goto 102);  $S.\text{nextlist} = B.\text{falselist} = \{101\}$ ; emit 106: goto 100.

```

100:  if a < b goto 102
101:  goto _           <- S.nextlist, patched by whoever follows
102:  if c < d goto 104
103:  goto 100
104:  t1 = y + z

```

```

105:  x = t1
106:  goto 100
107:  ...          <- (101 will be backpatched with 107)

```

**Worked example: if (a < b) x = 1; else x = 2;**

```

100:  if a < b goto 102      // B.truelist = {100}, patched to M1 = 102
101:  goto 104               // B.falselist = {101}, patched to M2 = 104
102:  x = 1
103:  goto _                // N: on S.nextlist
104:  x = 2
105:  ...                   // 103 is patched to 105 by the enclosing construct

```

## 8.4 Comparison: inherited labels vs. backpatching

|                 | Label-based SDD<br>(Fig. 6.36/6.37)                           | Backpatching (Fig. 6.43/6.46)                                              |
|-----------------|---------------------------------------------------------------|----------------------------------------------------------------------------|
| Attributes      | Inherited labels ( <i>true</i> , <i>false</i> , <i>next</i> ) | Synthesized lists ( <i>truelist</i> , <i>falselist</i> , <i>nextlist</i> ) |
| Class of SDD    | L-attributed                                                  | S-attributed (with markers <i>M</i> , <i>N</i> )                           |
| Best suited for | Top-down / recursive-descent (or AST traversal)               | Bottom-up (LR) parsing, single pass                                        |
| Code storage    | Attribute <i>.code</i> , or incremental <i>gen</i>            | Instruction array; patched later                                           |
| Labels          | Symbolic labels generated eagerly                             | Instruction indices known late                                             |

## 9 Switch Statements and Procedure Calls (Brief)

### 9.1 Switch statement

```

switch (E) {
  case V1: S1
  case V2: S2
  ...
  case V(n-1): S(n-1)
  default: Sn
}

```

**Translation scheme:** evaluate  $E$  into a temporary  $t$ , then a sequence of tests, followed by the code of each case.

```

      code to evaluate E into t
      goto test
L1:   code for S1
      goto next
L2:   code for S2
      goto next
...
Ln:   code for Sn
      goto next
test: if t = V1 goto L1
      if t = V2 goto L2

```

```

...
if t = V(n-1) goto L(n-1)
goto Ln
next:

```

Putting the tests *after* the case bodies lets the code generator recognise the test sequence and implement it with a **jump table** (for dense values) or binary search. The **case** instruction can also be used: **case** t V1 L1 ... **case** t V(n-1) L(n-1); **case** t t Ln (the last entry is the default).

### Break and fall-through

The scheme above puts **goto next** after every case body, which is the behaviour of Pascal-like **case** statements. In C and Java a body *falls through* into the next one unless it ends with **break**; to translate those, emit **goto next** only for **break**, and leave it out otherwise.

### Worked example

```

switch (c) {
  case 1: x = 10; break;
  case 2: x = 20; break;
  default: x = 0;
}

```

(The selector is a variable, so no code is needed to evaluate it and  $t = c$ .)

```

      goto test
L1:   x = 10
      goto next
L2:   x = 20
      goto next
L3:   x = 0
      goto next
test: if c = 1 goto L1
      if c = 2 goto L2
      goto L3
next:

```

Because the tests are collected at the end, the code generator can see that the values 1, 2, ... are dense and replace them by one indexed jump through a table  $[L_1, L_2]$  after the range check  $1 \leq c \leq 2$ .

## 9.2 Procedure calls

For a call  $id(E_1, E_2, \dots, E_n)$ :

```

code to evaluate E1 into t1
code to evaluate E2 into t2
...
code to evaluate En into tn
param t1
param t2
...
param tn
call id, n           // or: y = call id, n   if a value is returned

```

Grammar and actions (the parameter addresses are collected in a queue, and then emitted as **param**):

$$\begin{aligned}
 S &\rightarrow \text{call id} (Elist) \quad \{ \text{for each } t \text{ in queue: } gen(\text{param } t); gen(\text{call id.addr}, size(queue)); \} \\
 Elist &\rightarrow Elist, E \quad \{ \text{append } E.addr \text{ to the end of } queue; \} \\
 Elist &\rightarrow E \quad \{ queue = [E.addr]; \}
 \end{aligned}$$

### Worked example: $y = f(a + b, c * 2)$

The arguments are evaluated first, left to right, then the **param** instructions are emitted, and finally the call:

```

t1 = a + b
t2 = c * 2
param t1
param t2
y = call f, 2

```

### One global queue is not enough for nested calls

Take  $f(a, g(b))$ . After the first argument the queue of  $f$  is  $[a]$ . Parsing  $g(b)$  then runs  $Elist \rightarrow E$ , which *resets* the same queue to  $[b]$ , destroying  $f$ 's pending  $a$ . The fix is to keep the list as a synthesized attribute ( $Elist.list$ ), or equivalently to use a *stack* of queues. The correct code is

```

param b
t1 = call g, 1
param a
param t1
t2 = call f, 2

```

## 10 A Complete Worked Example

### Translating a small program fragment to three-address code

Source fragment (all variables are **int**; **a** is **int a[10]**, so  $w = 4$ ):

```

i = 0;
while (i < n && a[i] != key)
    i = i + 1;

```

**Using the label-based SDD.**  $P \rightarrow S$  gives  $S.next = L_0$ . For the sequence  $S_1 S_2$  (the assignment, then the loop) a label  $L_1 = S_1.next$  is placed between them. For the loop,  $begin = L_2$ ,  $B.true = L_3$ ,  $B.false = L_0$ , and the body's  $S_1.next = L_2$ . For  $B = B_1 \&\& B_2$  a new label  $L_4 = B_1.true$  is created, with  $B_1.false = L_0$ ,  $B_2.true = L_3$ ,  $B_2.false = L_0$ .

```

        i = 0
L1:
L2:     if i < n goto L4      // B1 (begin of while)
        goto L0              // B1 false -> exit loop
L4:     t1 = i * 4            // B2: a[i] != key
        t2 = a[t1]
        if t2 != key goto L3
        goto L0              // B false -> exit loop
L3:     t3 = i + 1           // loop body
        i = t3

```

```

        goto L2          // back to begin
L0:

```

(L1 and L2 label the same instruction; a later pass can merge them.)

**Using backpatching** (instruction numbering from 100). Here  $M_1.instr = 101$  (start of loop), the marker of `&&` gives 103, and  $M_2.instr = 107$  (start of body).

```

100:  i = 0
101:  if i < n goto 103      // patched by the && marker
102:  goto _                // on B.falselist = S.nextlist
103:  t1 = i * 4
104:  t2 = a[t1]
105:  if t2 != key goto 107  // truelist patched with M2 = 107
106:  goto _                // on B.falselist = S.nextlist
107:  t3 = i + 1
108:  i = t3
109:  goto 101              // emitted by the while rule (M1)
110:  ...                  // {102, 106} are backpatched with 110

```

Both of the incomplete jumps at 102 and 106 form  $S.nextlist$  of the loop; whoever follows the loop will patch them with the index of the next instruction, here 110.

## 11 Summary and Key Takeaways

- Intermediate code (syntax trees, DAGs, three-address code in quadruple/triple form) decouples front and back ends and enables machine-independent optimisation.
- Syntax alone does not give meaning. An **SDD** attaches *attributes* to grammar symbols and *semantic rules* to productions; **synthesized** attributes flow upward, **inherited** attributes flow downward or sideways. **S-attributed**  $\subset$  **L-attributed**.
- An **SDT** embeds the actions in the grammar and fixes the order of evaluation. S-attributed SDDs  $\rightarrow$  *postfix SDTs* (bottom-up). L-attributed SDDs can be implemented during top-down parsing, or, for LR grammars and with marker nonterminals, during bottom-up parsing.
- Expressions:  $E.addr$  holds the result; *gen* emits TAC; temporaries are created by *newTemp*. Array references use offsets computed from the widths of sub-arrays.
- Boolean expressions are translated into **jumping code**, using the inherited labels *true/false*. Statements use *next*.
- **Backpatching** replaces inherited labels by synthesized lists (*makelist*, *merge*, *backpatch*), allowing one-pass bottom-up generation with markers  $M$  and  $N$ .

### Common mistakes (check your own answers against these)

- Reading a *value number* as the value of the expression. It is only the index of a record.
- Forgetting that a DAG shares a node only if *operator and both children* match;  $b + a$  and  $a + b$  are different unless the operands are put in canonical order.
- Using  $B.true$  where  $S.next$  is meant:  $S.next$  is where control goes *after* a statement,  $B.true$  is where it goes *into* the body.
- In a postfix SDT, putting an action in the middle of a body and expecting an LR parser to run it there; use a marker.
- Off-by-one errors in backpatching:  $M.instr$  is the index of the *next* instruction to be emitted, not of the last one.

- Treating array subscripts as indices in the three-address code; they are scaled to byte offsets.
- Evaluating both operands of `&&` / `||` in jumping code, which loses short-circuit behaviour.

## 12 Practice Problems

- Q1.** Translate  $x = a * (b + c) - d / (b + c)$  into (a) a syntax tree and DAG, (b) three-address code, (c) quadruples, triples, and indirect triples.
- Q2.** Using the SDD of Table 5, annotate the parse tree and generate TAC for  $a = -(b + c) * (b + c)$ .
- Q3.** For `int a[5][4]` (int width = 4), write the TAC for  $x = a[i][j] + a[j][i]$ . Give the width used at each dimension.
- Q4.** Generate jumping code (with inherited labels) for `if (a < b && !(c < d)) x = 1; else x = 2;`
- Q5.** Show, step by step, the *truelist/false list* and the final code (with backpatching, starting at 100) for  $a < b \parallel c < d \&\& e < f$ .
- Q6.** Using backpatching, translate: `while (x < 10) { x = x + 1; if (x == 5) y = 0; }`
- Q7.** Classify as S-attributed, L-attributed, or neither: (a)  $A \rightarrow BC$  with  $C.i = B.s$  and  $A.s = C.s$ ; (b)  $A \rightarrow BC$  with  $B.i = C.s$ ; (c) the SDD of Table 5.
- Q8.** Write the SDT for  $E \rightarrow E_1 * E_2 \mid E_1 + E_2 \mid (E) \mid \text{id}$  that prints the *postfix* form, and show its output for  $a + b * (c + d)$ .
- Q9.** Build the array of value-number records for  $a * b + a * b + c$  and give the three-address code obtained from it.
- Q10.** Translate `switch (n) { case 1: s = 1; break; case 5: s = 2; break; default: s = 0; }` using the scheme of Section ??.
- Q11.** Let `x` be `float` and `i, j` be `int`. Using the widening rules, write the code for  $x = i + j * x$ .
- Q12.** Generate the fall-through jumping code for `if (a < b || c < d) x = 1;` starting with  $B.true = fall$ .

### Answers to selected problems

**Q1(b).** The DAG shares  $b + c$ , so the TAC is

```
t1 = b + c
t2 = a * t1
t3 = d / t1
t4 = t2 - t3
x = t4
```

**Q3.** Row width =  $4 \times 4 = 16$ ; element width = 4.

```
t1 = i * 16
t2 = j * 4
t3 = t1 + t2
t4 = a[t3]
t5 = j * 16
```

```

t6 = i * 4
t7 = t5 + t6
t8 = a[t7]
t9 = t4 + t8
x = t9

```

**Q4.**  $B.true = L_2$ ,  $B.false = L_3$ ,  $S.next = L_1$ ; for  $B_1 \&\& B_2$  with  $B_2 = !B_3$ :  $B_1.true = L_4$ ,  $B_3.true = L_3$ ,  $B_3.false = L_2$ .

```

        if a < b goto L4
        goto L3
L4:      if c < d goto L3
        goto L2
L2:      x = 1
        goto L1
L3:      x = 2
L1:

```

**Q7.** (a) L-attributed ( $C.i$  depends on the left sibling  $B$ ) but not S-attributed. (b) Neither (inherited attribute of  $B$  depends on the right sibling  $C$ ). (c) S-attributed (and hence L-attributed).

**Q8.** Postfix SDT:  $E \rightarrow E_1 * E_2 \{print('*')\}$ ,  $E \rightarrow E_1 + E_2 \{print('+')\}$ ,  $E \rightarrow (E)$ ,  $E \rightarrow id \{print(id.lexeme)\}$  (with precedence/associativity declared or via an unambiguous grammar). Output for  $a + b * (c + d)$ :  $a \ b \ c \ d \ + \ * \ +$ .

**Q2.** Unary minus binds tighter than  $*$ , so the expression is  $-(b+c)*(b+c)$ . The SDD (no DAG) computes  $b+c$  twice:

```

t1 = b + c
t2 = minus t1
t3 = b + c
t4 = t2 * t3
a = t4

```

**Q5.**  $B_1 \equiv a < b$  at 100–101,  $M_1 = 102$ ,  $B_3 \equiv c < d$  at 102–103,  $M_2 = 104$ ,  $B_4 \equiv e < f$  at 104–105. Reducing  $B_3 \&\& M_2 B_4$  patches  $\{102\}$  with 104 and gives  $truelist = \{104\}$ ,  $falselist = \{103, 105\}$ . Reducing  $B_1 || M_1 B_2$  patches  $\{101\}$  with 102 and gives  $truelist = \{100, 104\}$ ,  $falselist = \{103, 105\}$ .

```

100:  if a < b goto _      <- truelist
101:  goto 102
102:  if c < d goto 104
103:  goto _              <- falselist
104:  if e < f goto _      <- truelist
105:  goto _              <- falselist

```

**Q6.**  $M_1 = 100$ ; test at 100–101;  $M_2 = 102$ ; body: 102:  $t1 = x + 1$ , 103:  $x = t1$ ; inner marker 104; inner test 104–105; inner marker 106; 106:  $y = 0$ . The inner **if** has  $nextlist = \{105\}$ , which the **while** patches with  $M_1 = 100$ .

```

100:  if x < 10 goto 102
101:  goto _              <- S.nextlist (patched later)
102:  t1 = x + 1
103:  x = t1
104:  if x == 5 goto 106
105:  goto 100
106:  y = 0
107:  goto 100
108:  ...                <- 101 is patched with 108

```

**Q9.** Records: 1 id  $a$ ; 2 id  $b$ ; 3  $(*, 1, 2)$ ; (second  $a * b$  is found, value number 3); 4  $(+, 3, 3)$ ; 5 id  $c$ ;

6 (+, 4, 5). Code:  $t_1 = a * b$ ,  $t_2 = t_1 + t_1$ ,  $t_3 = t_2 + c$ .

**Q10.**

```
        goto test
L1:      s = 1
        goto next
L2:      s = 2
        goto next
L3:      s = 0
        goto next
test:    if n = 1 goto L1
        if n = 5 goto L2
        goto L3
next:
```

Here the values 1 and 5 are *sparse*, so the code generator would prefer a binary search or the comparison chain above to a jump table of five entries.

**Q11.**

```
t1 = (float) j
t2 = t1 * x
t3 = (float) i
t4 = t3 + t2
x  = t4
```

**Q12.**  $B.true = fall$ ,  $B.false = L_1$ ; for  $B_1 || B_2$ :  $B_1.true = L_2$  (new),  $B_1.false = fall$ ,  $B_2.true = fall$ ,  $B_2.false = L_1$ .

```
        if a < b goto L2
        ifFalse c < d goto L1
L2:      x = 1
L1:
```

## References

1. A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson, 2007. – Chapter 5 (Syntax-Directed Translation) and Chapter 6 (Intermediate-Code Generation).